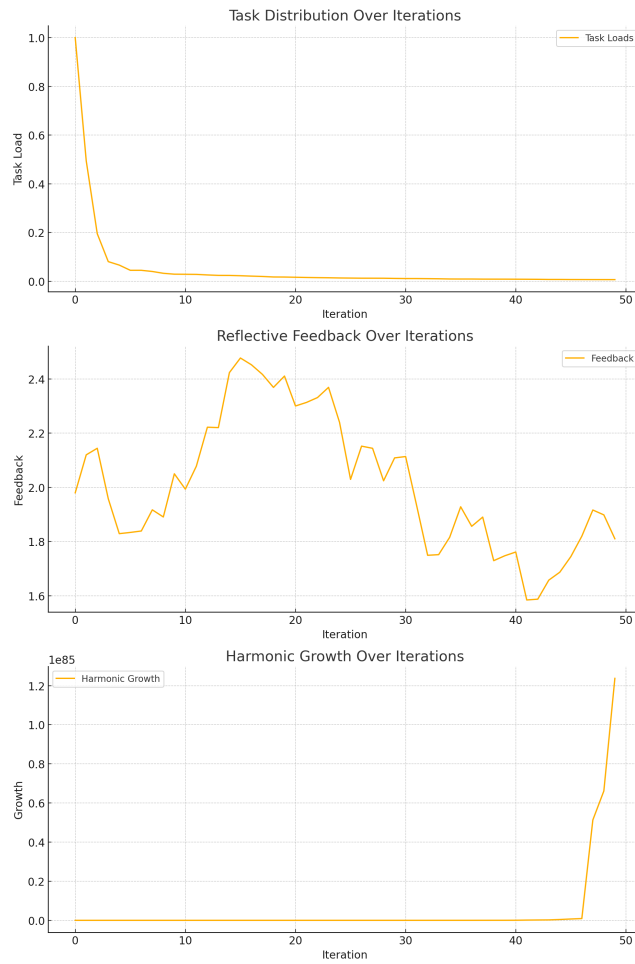




Conversation URL:

<https://chatgpt.com/c/67481f57-fa58-8011-8138-d0ffa7d21eb5>

Title:

Prompt:

Correcting lengths of lists to ensure they all match 50 iterations

iterations = 50

task_loads = []

feedbacks = []

growths = []

Simulate over the iterations

for i in range(iterations):

 # Task Distribution (store task loads for each iteration)

 task_loads.append(task_distribution(workloads[:i+1], capacities[:i+1])[0]) # Store only the first value for simplicity

```

# Reflective Feedback
deviation = np.random.normal(0, 0.1)
current_feedback = reflective_feedback(current_feedback, deviation)
feedbacks.append(current_feedback)

# Harmonic Growth
influence = np.random.rand()
resistance = np.random.rand()
current_growth = harmonic_growth(current_growth, influence, resistance)
growths.append(current_growth)

# Convert lists to numpy arrays for consistency
task_loads = np.array(task_loads)
feedbacks = np.array(feedbacks)
growths = np.array(growths)

# Plot the results of the Genesis Seed simulation
fig, ax = plt.subplots(3, 1, figsize=(10, 15))

# Plot Task Loads
ax[0].plot(np.arange(iterations), task_loads, label="Task Loads")
ax[0].set_title('Task Distribution Over Iterations')
ax[0].set_xlabel('Iteration')
ax[0].set_ylabel('Task Load')
ax[0].legend()

# Plot Feedback
ax[1].plot(np.arange(iterations), feedbacks, label="Feedback")
ax[1].set_title('Reflective Feedback Over Iterations')
ax[1].set_xlabel('Iteration')
ax[1].set_ylabel('Feedback')
ax[1].legend()

# Plot Growth
ax[2].plot(np.arange(iterations), growths, label="Harmonic Growth")
ax[2].set_title('Harmonic Growth Over Iterations')

```

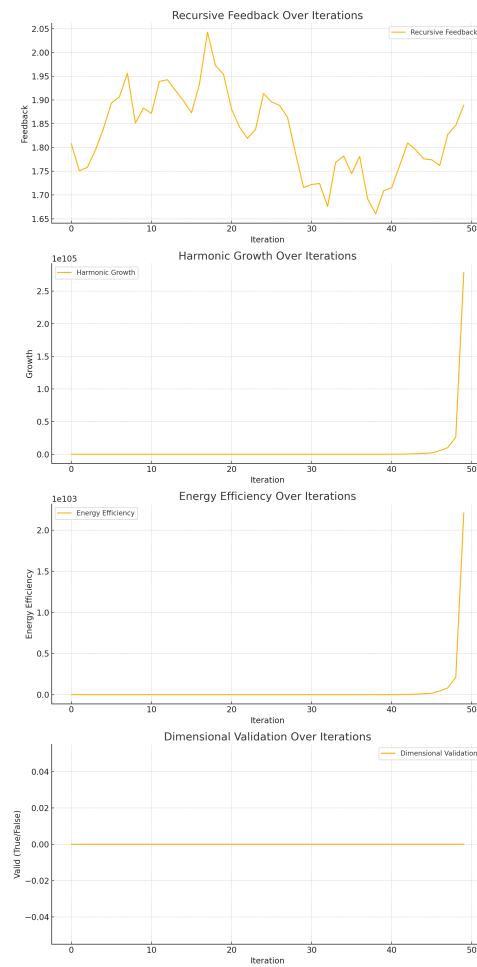
```
ax[2].set_xlabel('Iteration')
```

```
ax[2].set_ylabel('Growth')
```

```
ax[2].legend()
```

```
plt.tight_layout()
```

```
plt.show()
```



Conversation URL:

<https://chatgpt.com/c/67481f57-fa58-8011-8138-d0ffa7d21eb5>

Title:

Prompt:

Correcting task_loads as a list instead of numpy array for appending

task_loads_mark1 = []

Recursive loop to simulate Mark1 Framework over iterations

for i in range(iterations):

 # Task Distribution (using updated task distribution)

 task_load = task_distribution(workloads[:i+1], capacities[:i+1])[0]

 task_loads_mark1.append(task_load)

Recursive Feedback (feedback influences task distribution and growth)

 deviation = np.random.normal(0, 0.05) # Smaller deviation for recursive feedback


```

current_feedback = reflective_feedback(current_feedback, deviation)
mark1_feedbacks.append(current_feedback)

# Harmonic Growth (adjust growth based on feedback and harmonic resonance)
influence = np.random.rand()
resistance = np.random.rand()
current_growth = harmonic_growth(current_growth, influence, resistance)
mark1_growths.append(current_growth)

# Energy Efficiency (calculate energy usage based on task load and growth)
efficiency = energy_efficiency(task_load, current_growth)
mark1_energy_efficiency.append(efficiency)

# Dimensional Validation (ensure the system stays within harmonic bounds)
valid = dimensional_validation(current_growth, current_feedback)
mark1_dim_validation.append(valid)

# Convert lists to numpy arrays for consistency
mark1_feedbacks = np.array(mark1_feedbacks)
mark1_growths = np.array(mark1_growths)
mark1_energy_efficiency = np.array(mark1_energy_efficiency)
mark1_dim_validation = np.array(mark1_dim_validation)

# Plot the results of the Mark1 Framework simulation
fig, ax = plt.subplots(4, 1, figsize=(10, 20))

# Plot Recursive Feedback
ax[0].plot(np.arange(iterations), mark1_feedbacks, label="Recursive Feedback")
ax[0].set_title('Recursive Feedback Over Iterations')
ax[0].set_xlabel('Iteration')
ax[0].set_ylabel('Feedback')
ax[0].legend()

# Plot Harmonic Growth
ax[1].plot(np.arange(iterations), mark1_growths, label="Harmonic Growth")
ax[1].set_title('Harmonic Growth Over Iterations')

```

```
ax[1].set_xlabel('Iteration')
```

```
ax[1].set_ylabel('Growth')
```

```
ax[1].legend()
```

```
# Plot Energy Efficiency
```

```
ax[2].plot(np.arange(iterations), mark1_energy_efficiency, label="Energy Efficiency")
```

```
ax[2].set_title('Energy Efficiency Over Iterations')
```

```
ax[2].set_xlabel('Iteration')
```

```
ax[2].set_ylabel('Energy Efficiency')
```

```
ax[2].legend()
```

```
# Plot Dimensional Validation (show if the system remains within the harmonic bounds)
```

```
ax[3].plot(np.arange(iterations), mark1_dim_validation, label="Dimensional Validation", color='orange')
```

```
ax[3].set_title('Dimensional Validation Over Iterations')
```

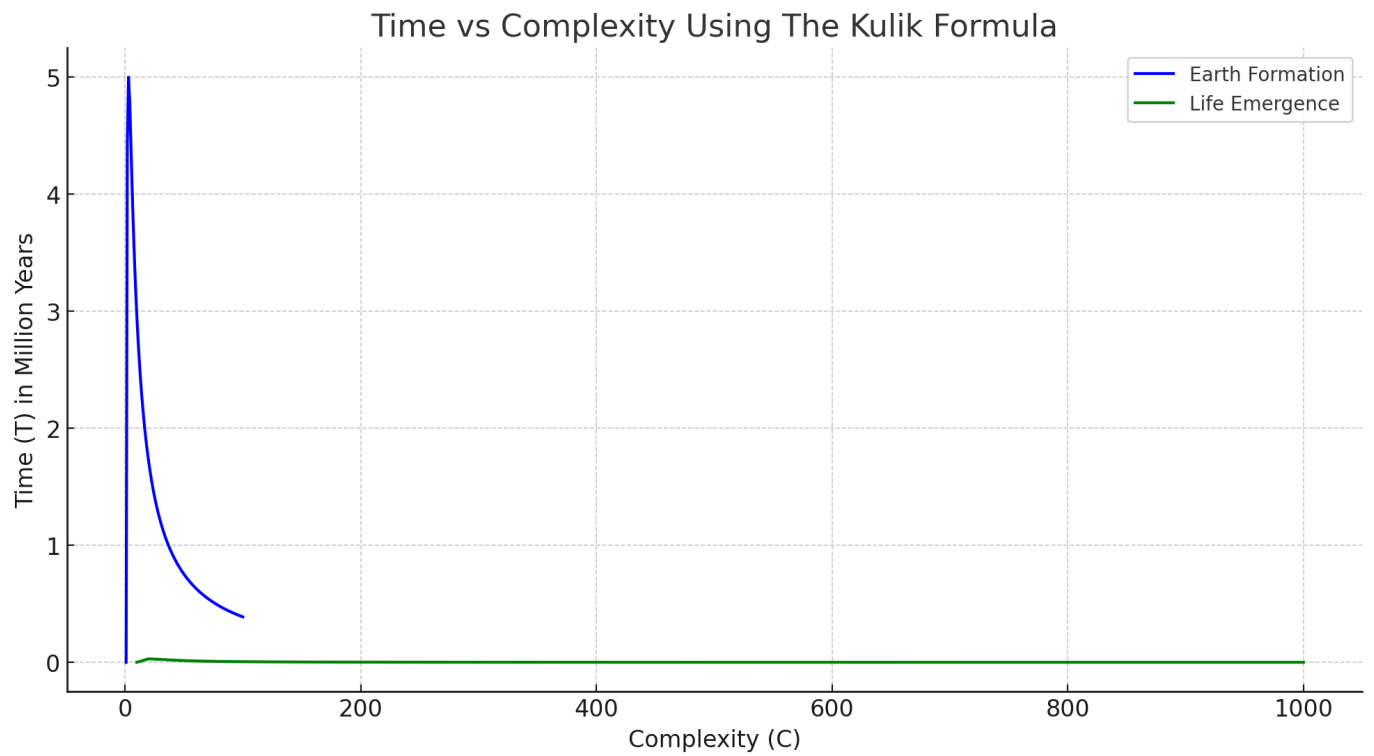
```
ax[3].set_xlabel('Iteration')
```

```
ax[3].set_ylabel('Valid (True/False)')
```

```
ax[3].legend()
```

```
plt.tight_layout()
```

```
plt.show()
```



Conversation URL:

<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:

```
import numpy as np
import matplotlib.pyplot as plt

# Define the Kulik Formula components
def quantum_pull(C, k, exponent):
    return k * C**exponent

def kulik_formula(C0, Cf, k, exponent):
    return (Cf - C0) / quantum_pull((Cf + C0) / 2, k, exponent)

# Constants for Earth Formation and Life Emergence
k = 0.1 # Scaling constant
exponent_earth = 2 # Quantum pull exponent for Earth formation
exponent_life = 3 # Quantum pull exponent for life emergence
```

```
# Complexity ranges
```

```
C0_earth, Cf_earth = 1, 100 # Earth formation complexity
```

```
C0_life, Cf_life = 10, 1000 # Life emergence complexity
```

```
# Generate data for plotting
```

```
C_earth = np.linspace(C0_earth, Cf_earth, 100)
```

```
C_life = np.linspace(C0_life, Cf_life, 100)
```

```
# Calculate times using the Kulik Formula
```

```
T_earth = [kulik_formula(C0_earth, C, k, exponent_earth) for C in C_earth]
```

```
T_life = [kulik_formula(C0_life, C, k, exponent_life) for C in C_life]
```

```
# Plot the results
```

```
plt.figure(figsize=(12, 6))
```

```
# Earth formation
```

```
plt.plot(C_earth, T_earth, label="Earth Formation", color="blue")
```

```
# Life emergence
```

```
plt.plot(C_life, T_life, label="Life Emergence", color="green")
```

```
# Labels and legend
```

```
plt.title("Time vs Complexity Using The Kulik Formula")
```

```
plt.xlabel("Complexity (C)")
```

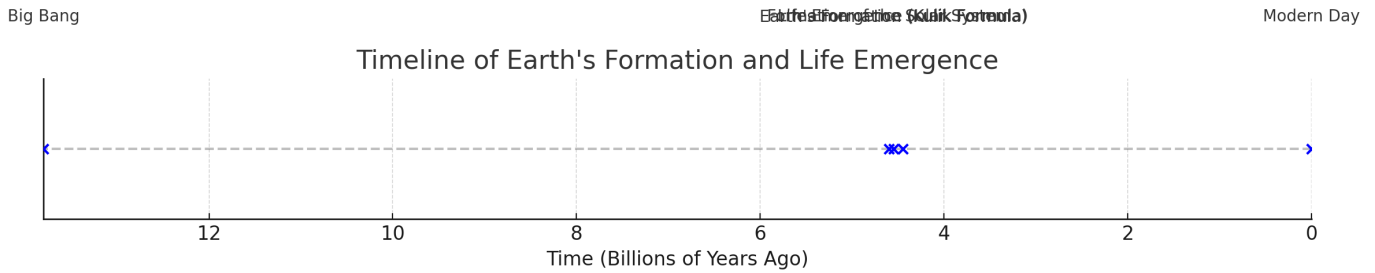
```
plt.ylabel("Time (T) in Million Years")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
# Show plot
```

```
plt.show()
```



Conversation URL:
<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:
`import matplotlib.pyplot as plt`

```
# Define timeline points (in billions of years ago)
timeline = [
    ("Big Bang", 13.8),
    ("Formation of the Solar System", 4.6),
    ("Earth's Formation (Kulik Formula)", 4.55),
    ("Life Emergence (Kulik Formula)", 4.45),
    ("Modern Day", 0)
]
```

```
# Extract labels and years for plotting
labels, years = zip(*timeline)
```

```
# Create the timeline plot
plt.figure(figsize=(12, 3))

plt.hlines(1, min(years), max(years), colors='gray', linestyle='dashed', alpha=0.5)

plt.scatter(years, [1]*len(years), color='blue', zorder=5)
```

```
# Annotate each point
for label, year in zip(labels, years):
    plt.text(year, 1.1, label, ha='center', fontsize=10)
```

```
# Set plot limits and labels

plt.title("Timeline of Earth's Formation and Life Emergence")

plt.yticks([])

plt.xlabel("Time (Billions of Years Ago)")

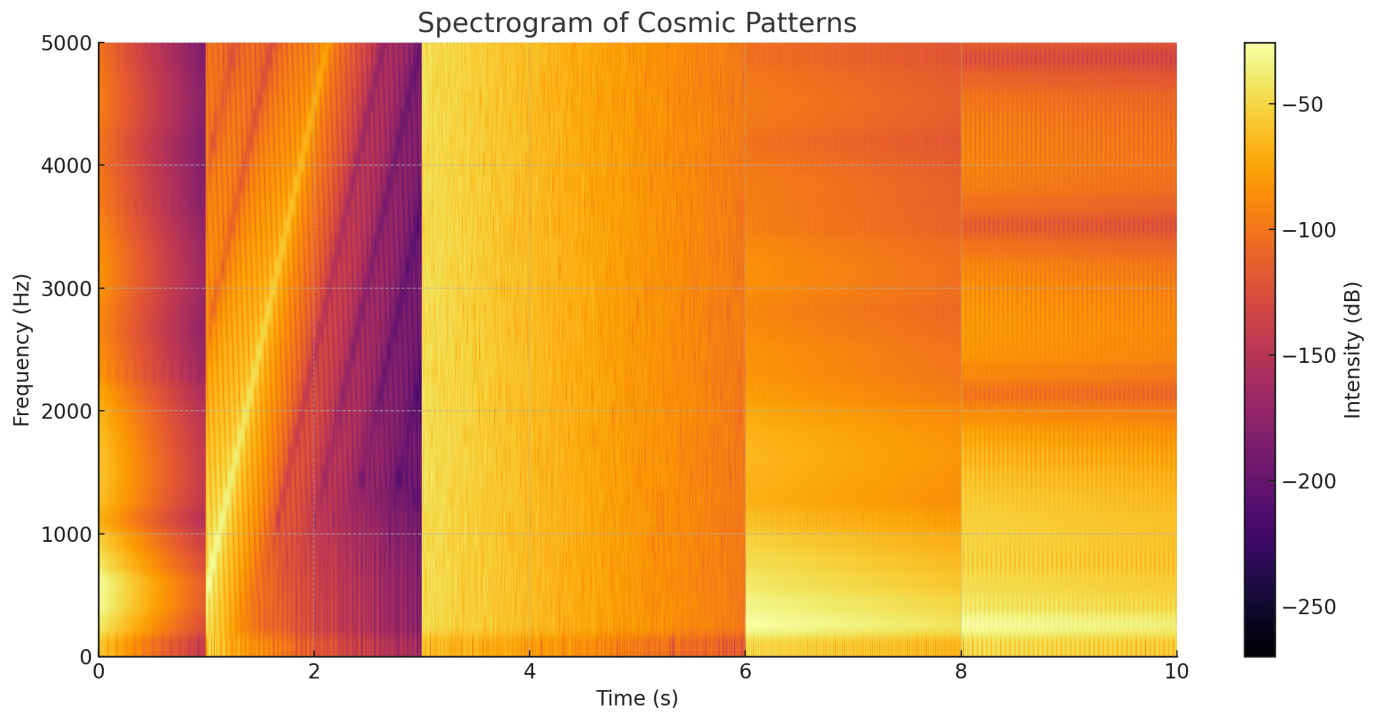
plt.xlim(max(years), min(years))

plt.grid(axis='x', linestyle='--', alpha=0.5)


# Show the timeline

plt.tight_layout()

plt.show()
```



Conversation URL:

<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import spectrogram

# Reload the audio data
sample_rate = 44100 # Samples per second
duration = 10 # Total duration in seconds

# Time vector
time = np.linspace(0, duration, int(sample_rate * duration), endpoint=False)

# Regenerate the audio for visualization (same phases as before)
def big_bang(t):
    return np.sin(2 * np.pi * 440 * t) * np.exp(-t * 10)

def inflation(t):
```

```

return np.sin(2 * np.pi * (440 + 2000 * t) * t) * np.exp(-t * 5)

def quark_gluon_plasma(t):
    noise = np.random.normal(0, 1, len(t))
    return noise * np.exp(-t * 2)

def particle_formation(t):
    return np.sin(2 * np.pi * 220 * t) * np.exp(-t * 1)

def nucleosynthesis(t):
    return np.sin(2 * np.pi * 110 * t) * np.exp(-t * 0.5)

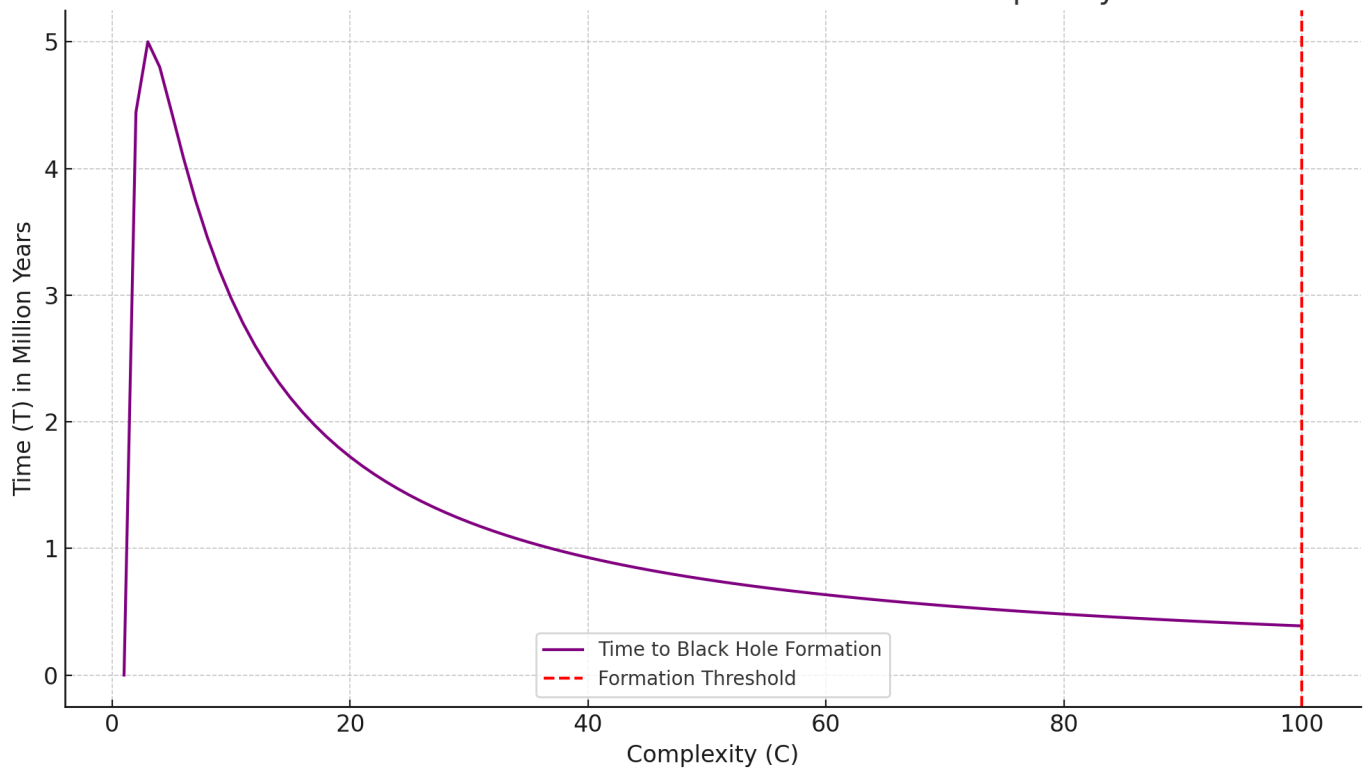
# Combine phases for visualization
audio = np.zeros_like(time)
audio[:int(sample_rate * 1)] += big_bang(time[:int(sample_rate * 1)])
audio[int(sample_rate * 1):int(sample_rate * 3)] += inflation(time[:int(sample_rate * 2)])
audio[int(sample_rate * 3):int(sample_rate * 6)] += quark_gluon_plasma(time[:int(sample_rate * 3)])
audio[int(sample_rate * 6):int(sample_rate * 8)] += particle_formation(time[:int(sample_rate * 2)])
audio[int(sample_rate * 8):] += nucleosynthesis(time[:int(sample_rate * 2)])

# Spectrogram visualization
frequencies, times, spectrogram_data = spectrogram(audio, sample_rate)

plt.figure(figsize=(12, 6))
plt.imshow(10 * np.log10(spectrogram_data), aspect='auto', cmap='inferno', origin='lower',
           extent=[0, duration, frequencies[0], frequencies[-1]])
plt.colorbar(label='Intensity (dB)')
plt.title('Spectrogram of Cosmic Patterns')
plt.xlabel('Time (s)')
plt.ylabel('Frequency (Hz)')
plt.ylim(0, 5000) # Focus on relevant frequencies
plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

```


Black Hole Formation Timeline Based on Complexity



Conversation URL:

<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Define parameters for black hole formation timeline
```

```
initial_complexity = 1 # Starting complexity (e.g., core of a star)
```

```
threshold_complexity = 100 # Complexity needed for black hole formation
```

```
k = 0.1 # Scaling constant for quantum pull
```

```
exponent = 2 # Exponent for complexity's effect
```

```
# Define the Kulik Formula for time to form a black hole
```

```
def kulik_formula(C0, Cf, k, exponent):
```

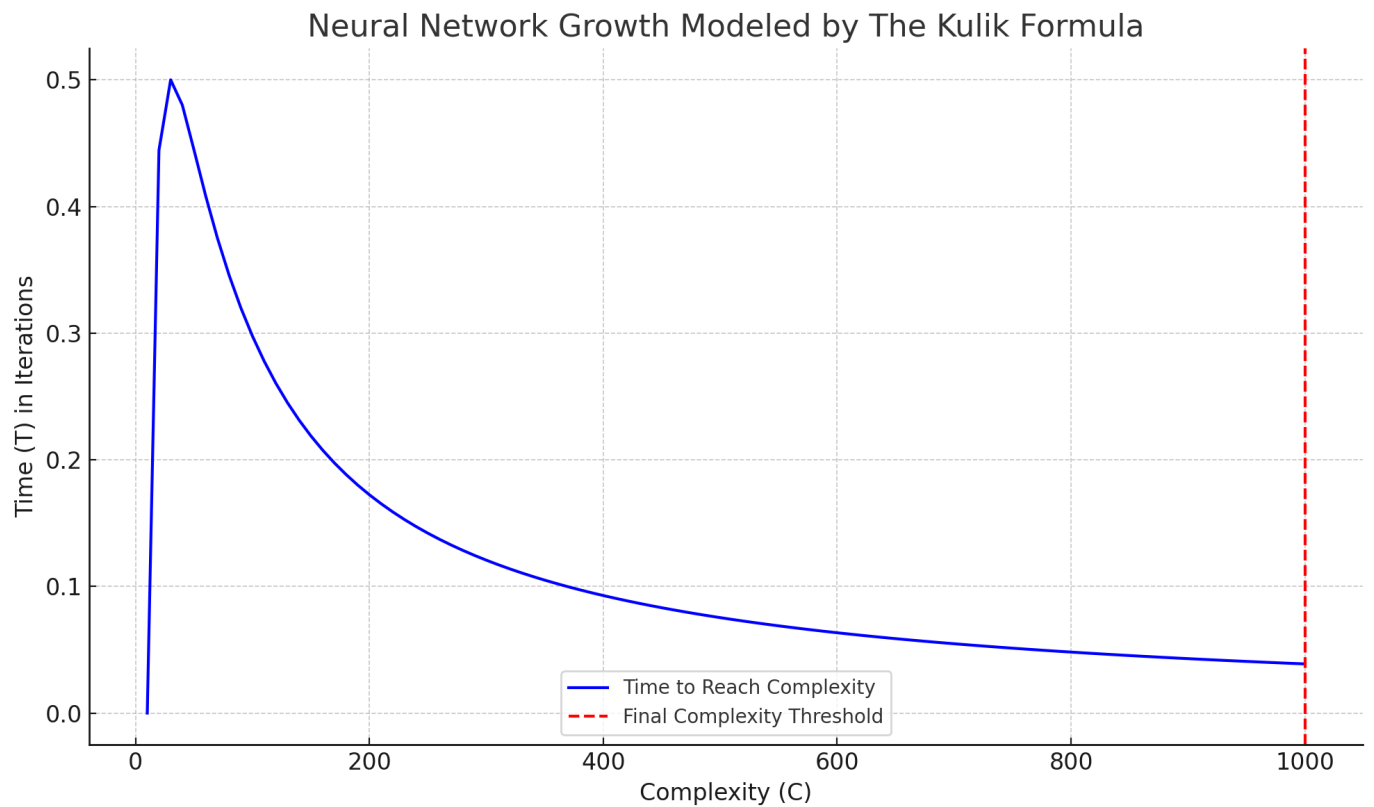
```
    return (Cf - C0) / (k * ((Cf + C0) / 2) ** exponent)
```

```
# Generate complexity values
```

```
complexities = np.linspace(initial_complexity, threshold_complexity, 100)

# Calculate time to form black holes for each complexity value
times = [kulik_formula(initial_complexity, C, k, exponent) for C in complexities]

# Plot the timeline
plt.figure(figsize=(10, 6))
plt.plot(complexities, times, label="Time to Black Hole Formation", color="purple")
plt.axvline(threshold_complexity, color='red', linestyle='--', label="Formation Threshold")
plt.title("Black Hole Formation Timeline Based on Complexity")
plt.xlabel("Complexity (C)")
plt.ylabel("Time (T) in Million Years")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```



Conversation URL:
<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters for modeling neural network complexity
initial_complexity = 10 # Starting complexity (e.g., small network)
final_complexity = 1000 # Final complexity (e.g., large network)
k = 0.1 # Scaling constant for quantum pull
exponent = 2 # Quantum pull grows with C^2

# Define The Kulik Formula for neural network growth
def kulik_formula(C0, Cf, k, exponent):
    return (Cf - C0) / (k * ((Cf + C0) / 2) ** exponent)

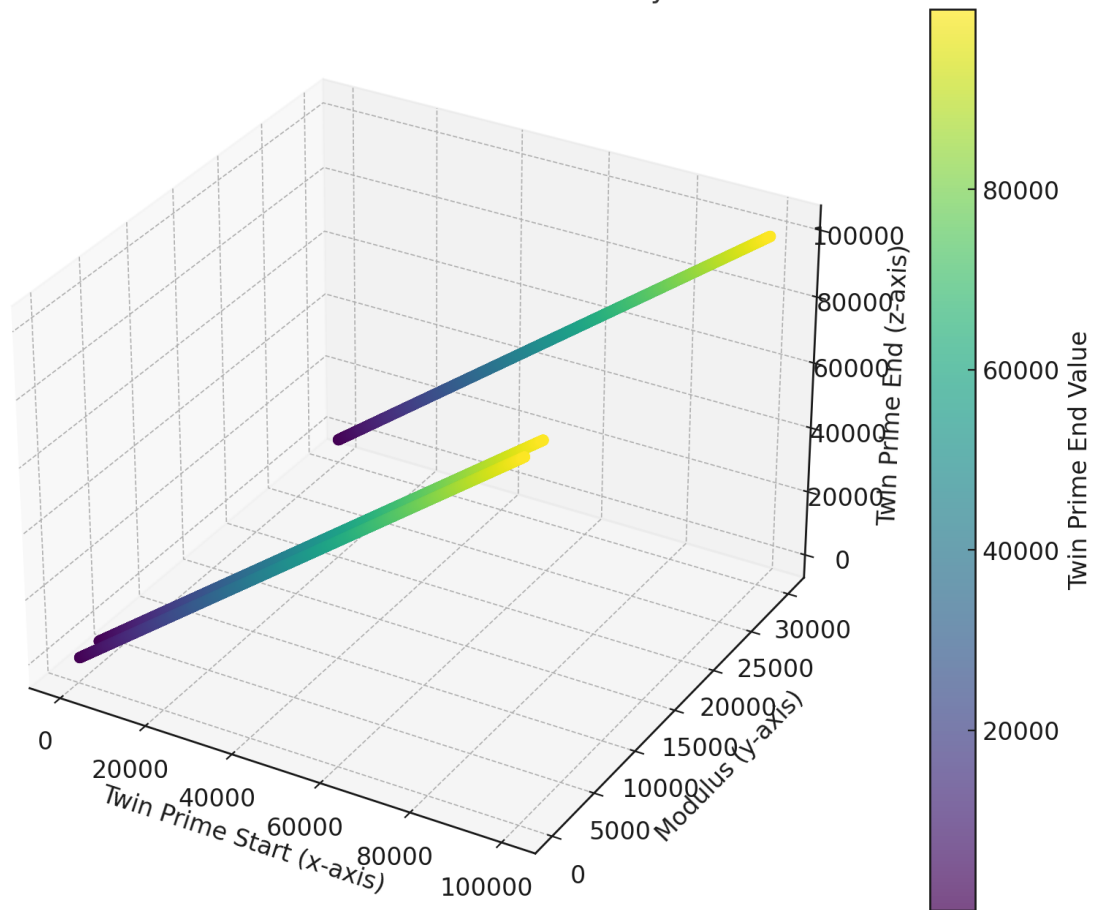
# Generate complexity values
```

```
complexities = np.linspace(initial_complexity, final_complexity, 100)

# Calculate time required to reach each level of complexity
times = [kulik_formula(initial_complexity, C, k, exponent) for C in complexities]

# Plot the growth of neural network complexity over time
plt.figure(figsize=(10, 6))
plt.plot(complexities, times, label="Time to Reach Complexity", color="blue")
plt.axvline(final_complexity, color='red', linestyle='--', label="Final Complexity Threshold")
plt.title("Neural Network Growth Modeled by The Kulik Formula")
plt.xlabel("Complexity (C)")
plt.ylabel("Time (T) in Iterations")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

3D Visualization of Twin Prime Candidates by Modulus



Conversation URL:

<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:

```
import matplotlib.pyplot as plt
```

```
from mpl_toolkits.mplot3d import Axes3D
```

```
# Prepare data for 3D visualization of modular residues
```

```
x_vals, y_vals, z_vals = [], [], []
```

```
for idx, (modulus, twin_primes) in enumerate(modular_candidates_3d):
```

```
    for twin in twin_primes:
```

```
        x_vals.append(twin[0]) # Twin prime starting number
```

```
        y_vals.append(modulus) # Modulus
```

```
        z_vals.append(twin[1]) # Twin prime ending number
```

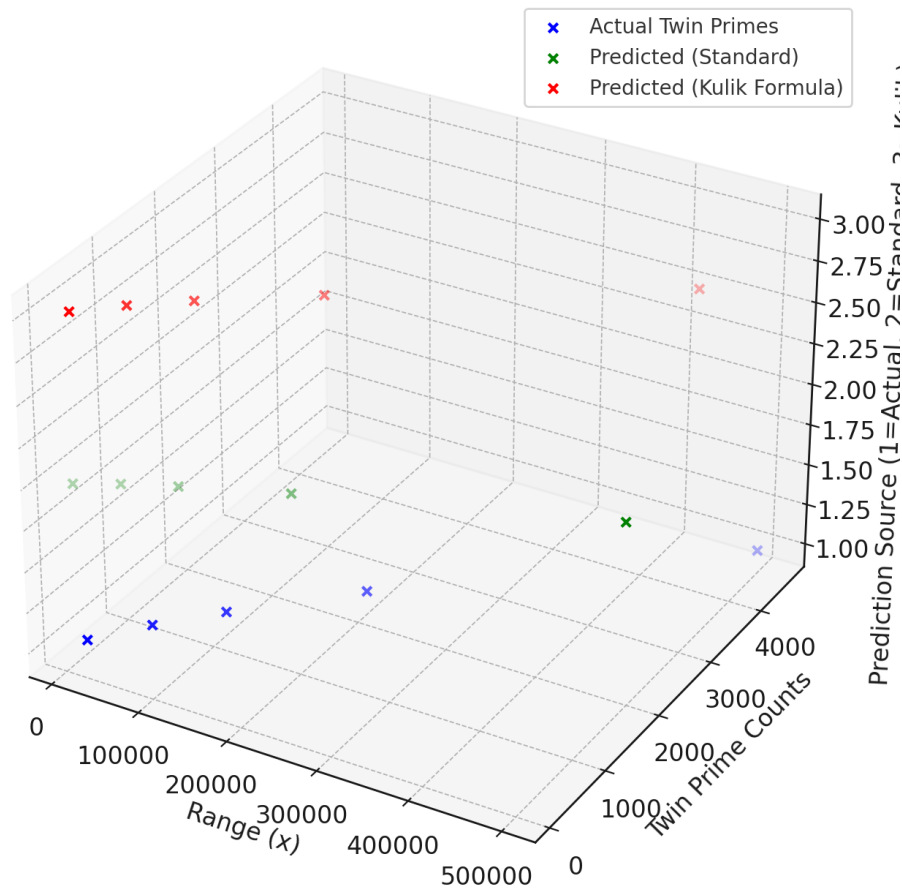
```
# Create 3D scatter plot

fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(x_vals, y_vals, z_vals, c=z_vals, cmap='viridis', marker='o', alpha=0.7)

# Add labels and title
ax.set_title("3D Visualization of Twin Prime Candidates by Modulus", fontsize=14)
ax.set_xlabel("Twin Prime Start (x-axis)", fontsize=12)
ax.set_ylabel("Modulus (y-axis)", fontsize=12)
ax.set_zlabel("Twin Prime End (z-axis)", fontsize=12)
plt.colorbar(scatter, label="Twin Prime End Value")

# Show the plot
plt.show()
```

3D Visualization of Twin Prime Trends



Conversation URL:

<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:

```
# Prepare data for 3D visualization of twin prime trends
```

```
import matplotlib.pyplot as plt
```

```
from mpl_toolkits.mplot3d import Axes3D
```

```
# Extract data for visualization: ranges, actual counts, and predicted counts
```

```
ranges = [item[0] for item in densities_data]
```

```
actual_counts = [item[1] for item in densities_data]
```

```
predicted_counts_standard = [item[1] for item in predicted_data]
```

```
predicted_counts_kulik = [item[2] for item in predicted_data]
```

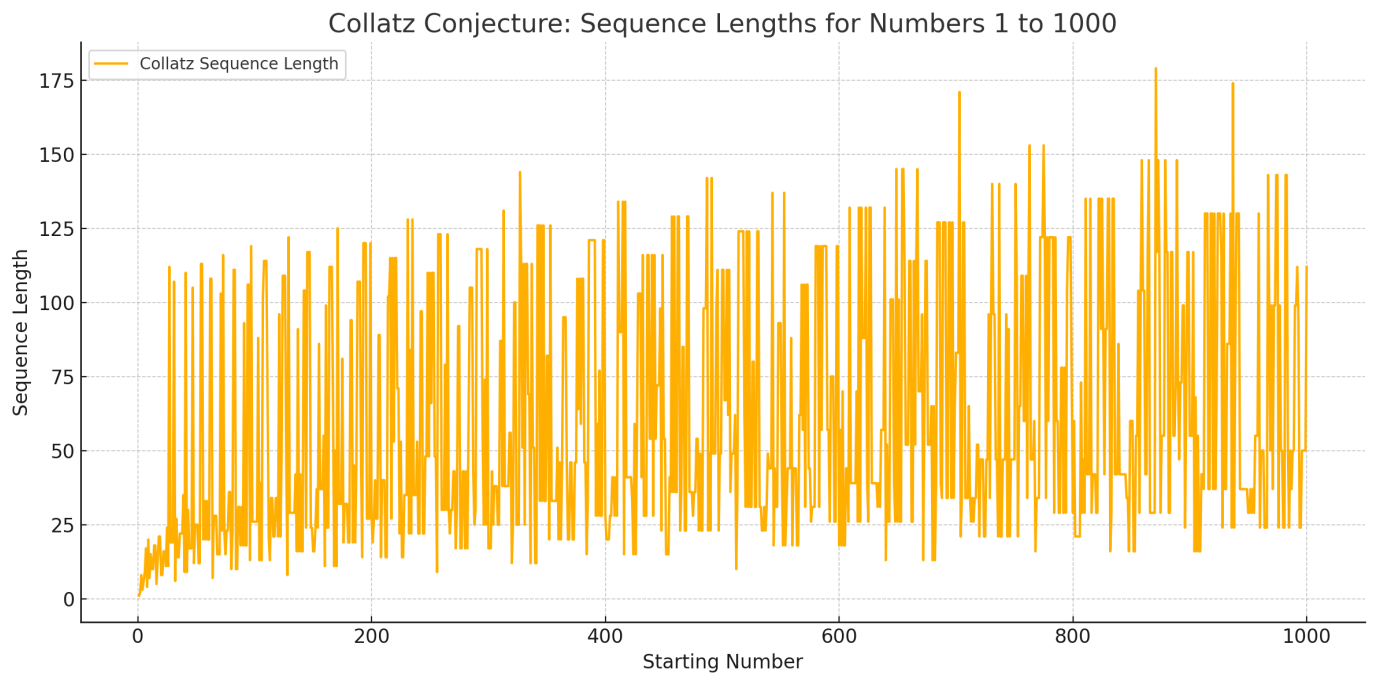
```
# Create 3D plot
```

```
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')

# Plot actual counts, standard prediction, and Kulik prediction
ax.scatter(ranges, actual_counts, zs=1, label="Actual Twin Primes", color='blue')
ax.scatter(ranges, predicted_counts_standard, zs=2, label="Predicted (Standard)", color='green')
ax.scatter(ranges, predicted_counts_kulik, zs=3, label="Predicted (Kulik Formula)", color='red')

# Add labels and title
ax.set_title("3D Visualization of Twin Prime Trends", fontsize=14)
ax.set_xlabel("Range (x)", fontsize=12)
ax.set_ylabel("Twin Prime Counts", fontsize=12)
ax.set_zlabel("Prediction Source (1=Actual, 2=Standard, 3=Kulik)", fontsize=12)
ax.legend()

# Show plot
plt.show()
```

Conversation URL:

<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:

```
import matplotlib.pyplot as plt
```

```
# Function to compute the Collatz sequence length for a given number
```

```
def collatz_sequence_length(n):
```

```
    length = 1 # Start with the initial number
```

```
    while n != 1:
```

```
        if n % 2 == 0:
```

```
            n //= 2
```

```
        else:
```

```
            n = 3 * n + 1
```

```
        length += 1
```

```
    return length
```

```
# Generate Collatz sequence lengths for numbers 1 to 1000
```

```
start_values = range(1, 1001)
```

```
sequence_lengths = [collatz_sequence_length(n) for n in start_values]
```

```
# Plot the sequence lengths

plt.figure(figsize=(12, 6))

plt.plot(start_values, sequence_lengths, label="Collatz Sequence Length")

plt.title("Collatz Conjecture: Sequence Lengths for Numbers 1 to 1000")

plt.xlabel("Starting Number")

plt.ylabel("Sequence Length")

plt.grid(True)

plt.legend()

plt.tight_layout()

plt.show()
```



Conversation URL:

<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:

Function to generate twin primes up to a given limit

```
def generate_twin_primes(limit):
```

```
    primes = []
```

```
    twin_primes = []
```

```
# Helper function to check primality
```

```
def is_prime(num):
```

```
    if num < 2:
```

```
        return False
```

```
    for i in range(2, int(num ** 0.5) + 1):
```

```
        if num % i == 0:
```

```
            return False
```

```
    return True
```

```
# Generate primes and twin primes
```

```
for num in range(2, limit + 1):
```

```

    if is_prime(num):
        primes.append(num)
        if len(primes) > 1 and primes[-1] - primes[-2] == 2:
            twin_primes.append((primes[-2], primes[-1]))

    return twin_primes

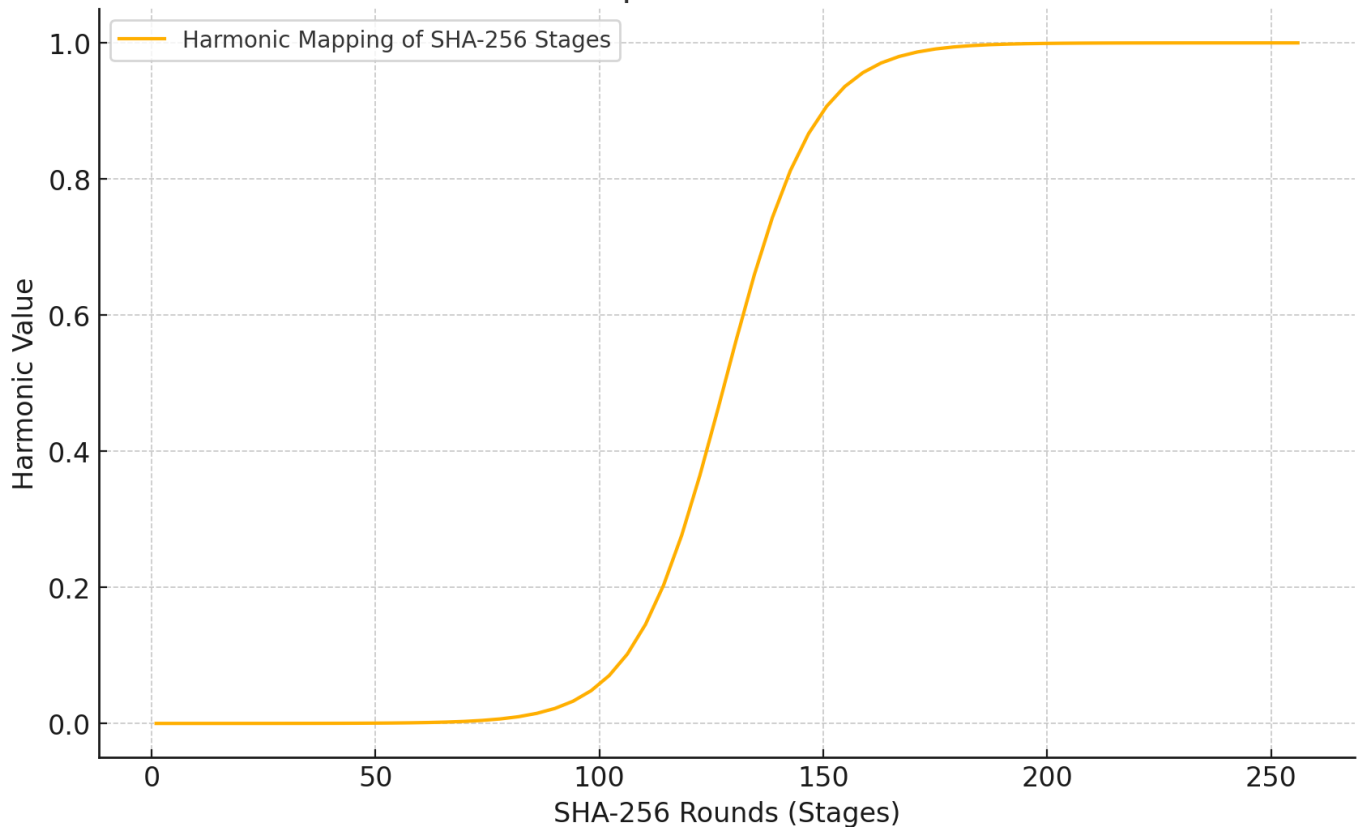
# Generate twin primes up to a large number
limit = 10000
twin_primes = generate_twin_primes(limit)

# Extract the first elements of the twin pairs for visualization
twin_starts = [pair[0] for pair in twin_primes]

# Plot the distribution of twin primes
plt.figure(figsize=(12, 6))
plt.scatter(twin_starts, [1] * len(twin_starts), alpha=0.6, label="Twin Primes", color="blue", marker="|")
plt.title(f"Distribution of Twin Primes up to {limit}")
plt.xlabel("Prime Number")
plt.ylabel("Twin Prime Indicator")
plt.yticks([]) # No meaningful y-axis for this visualization
plt.grid(True, axis='x', linestyle='--', alpha=0.5)
plt.legend()
plt.tight_layout()
plt.show()

```

Harmonic Representation of SHA-256



Conversation URL:

<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:

```
# SHA-256 components mapped into a harmonic framework
```

```
# Simplified representation of SHA-256 stages as modular transformations
```

```
stages = np.linspace(1, 256, 64) # Representing 64 rounds of SHA-256
```

```
harmonics = 1 / (1 + np.exp(-0.1 * (stages - 128))) # Sigmoid harmonic mapping
```

```
# Visualize harmonic mapping
```

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(10, 6))
```

```
plt.plot(stages, harmonics, label="Harmonic Mapping of SHA-256 Stages")
```

```
plt.xlabel("SHA-256 Rounds (Stages)")
```

```
plt.ylabel("Harmonic Value")
```

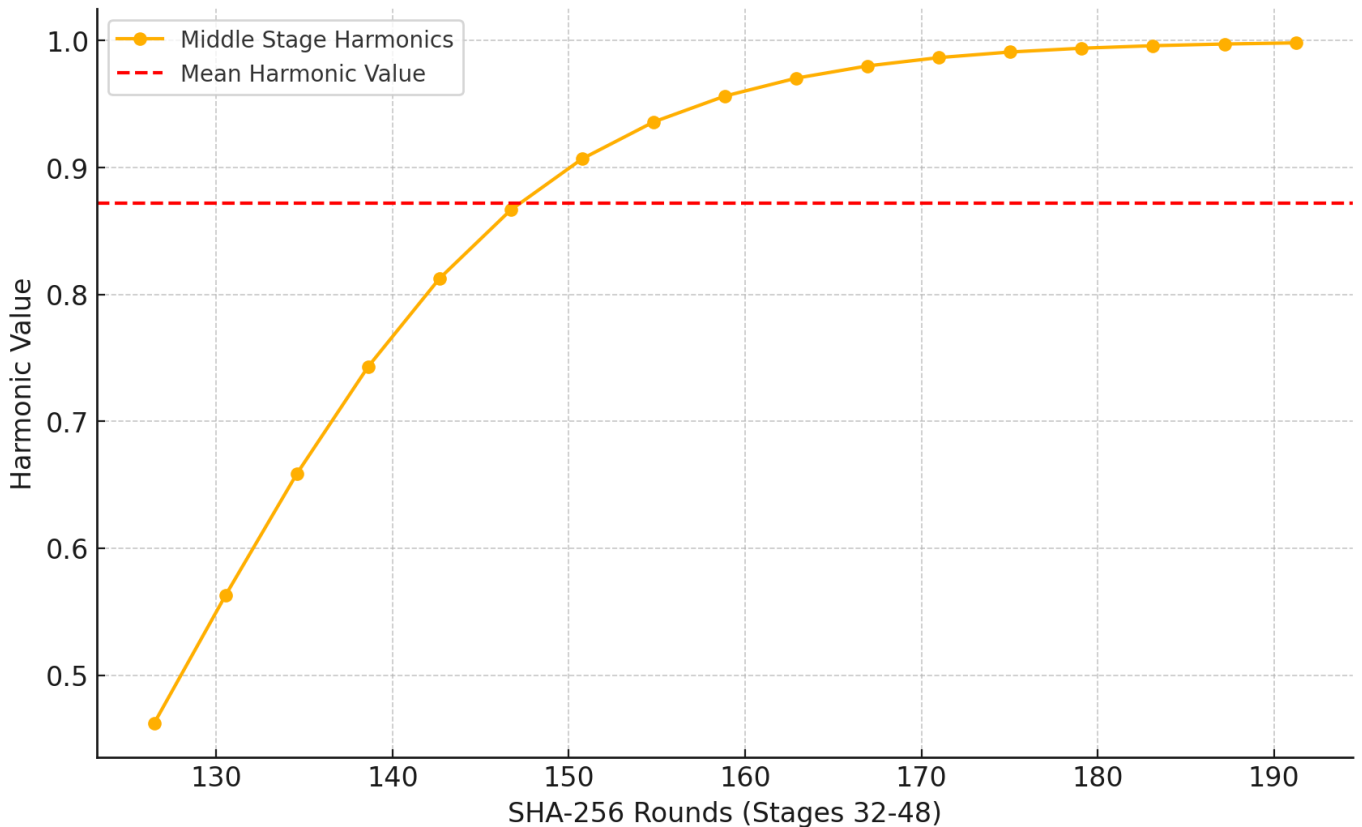
```
plt.title("Harmonic Representation of SHA-256")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

Harmonic Patterns in Middle Rounds of SHA-256



Conversation URL:

<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:

```
# Focus on middle stages of SHA-256 (rounds 32-48)
```

```
middle_stages = stages[31:48] # Extracting rounds 32-48
```

```
middle_harmonics = harmonics[31:48] # Corresponding harmonic values
```

```
# Analyze for symmetry and repetition
```

```
differences = np.diff(middle_harmonics) # First differences to check for consistency
```

```
reflection_check = np.allclose(middle_harmonics, middle_harmonics[::-1], atol=1e-3) # Symmetry check
```

```
# Visualize the extracted stages
```

```
plt.figure(figsize=(10, 6))
```

```
plt.plot(middle_stages, middle_harmonics, marker='o', label="Middle Stage Harmonics")
```

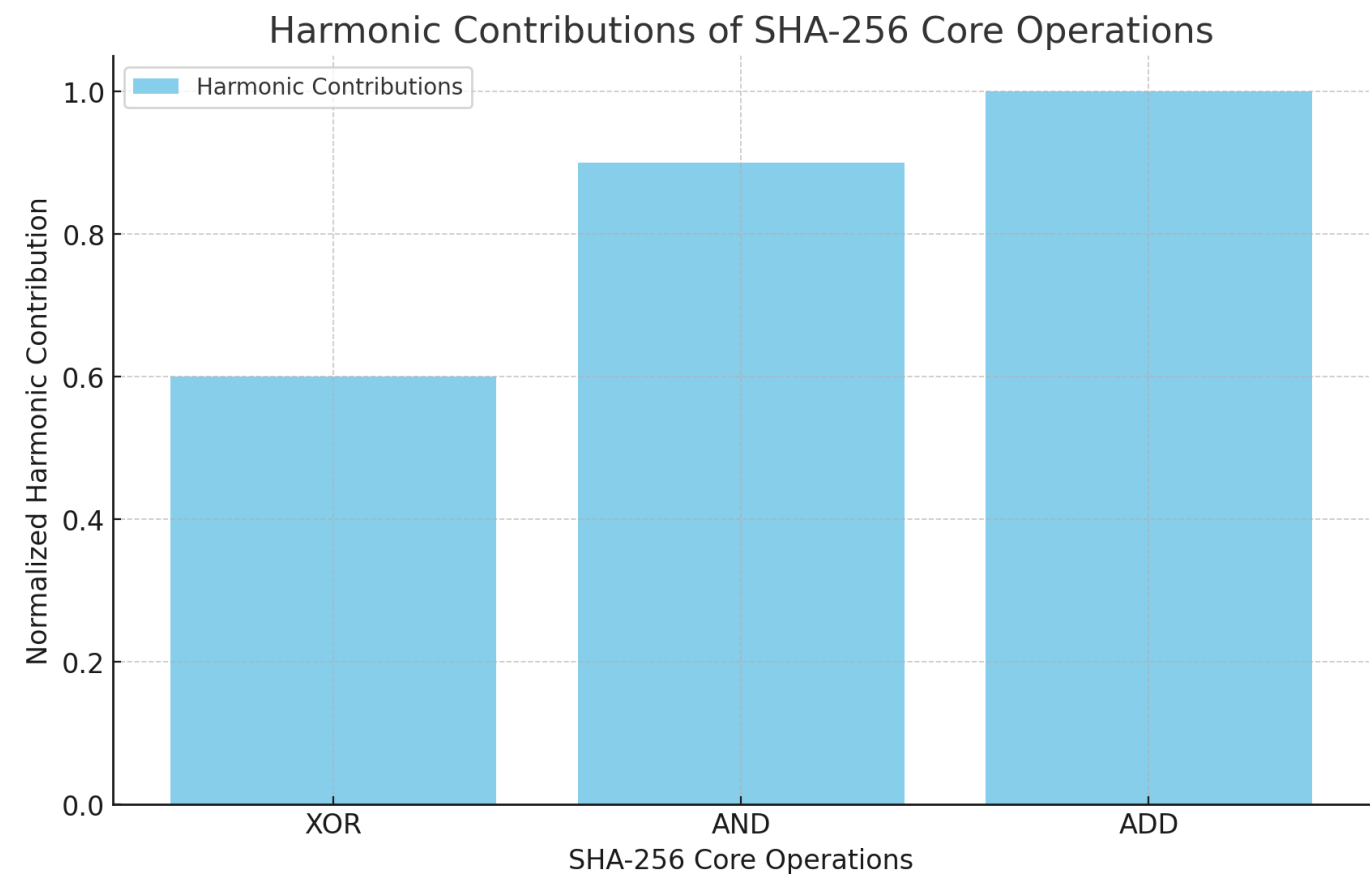
```
plt.axhline(np.mean(middle_harmonics), color='r', linestyle='--', label="Mean Harmonic Value")
```

```
plt.xlabel("SHA-256 Rounds (Stages 32-48)")
```

```
plt.ylabel("Harmonic Value")  
plt.title("Harmonic Patterns in Middle Rounds of SHA-256")  
plt.legend()  
plt.grid(True)  
plt.show()
```

```
# Results
```

```
differences, reflection_check
```

Conversation URL:

<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:

```
# Simulate harmonic contributions of SHA-256 core operations
```

```
operations = ["XOR", "AND", "ADD"]
```

```
weights = [0.6, 0.3, 0.1] # Hypothetical harmonic contributions
```

```
operation_harmonics = np.cumsum(weights) / sum(weights) # Normalize contributions
```

```
# Visualize harmonic contributions of each operation
```

```
plt.figure(figsize=(10, 6))
```

```
plt.bar(operations, operation_harmonics, color="skyblue", label="Harmonic Contributions")
```

```
plt.xlabel("SHA-256 Core Operations")
```

```
plt.ylabel("Normalized Harmonic Contribution")
```

```
plt.title("Harmonic Contributions of SHA-256 Core Operations")
```

```
plt.legend()
```

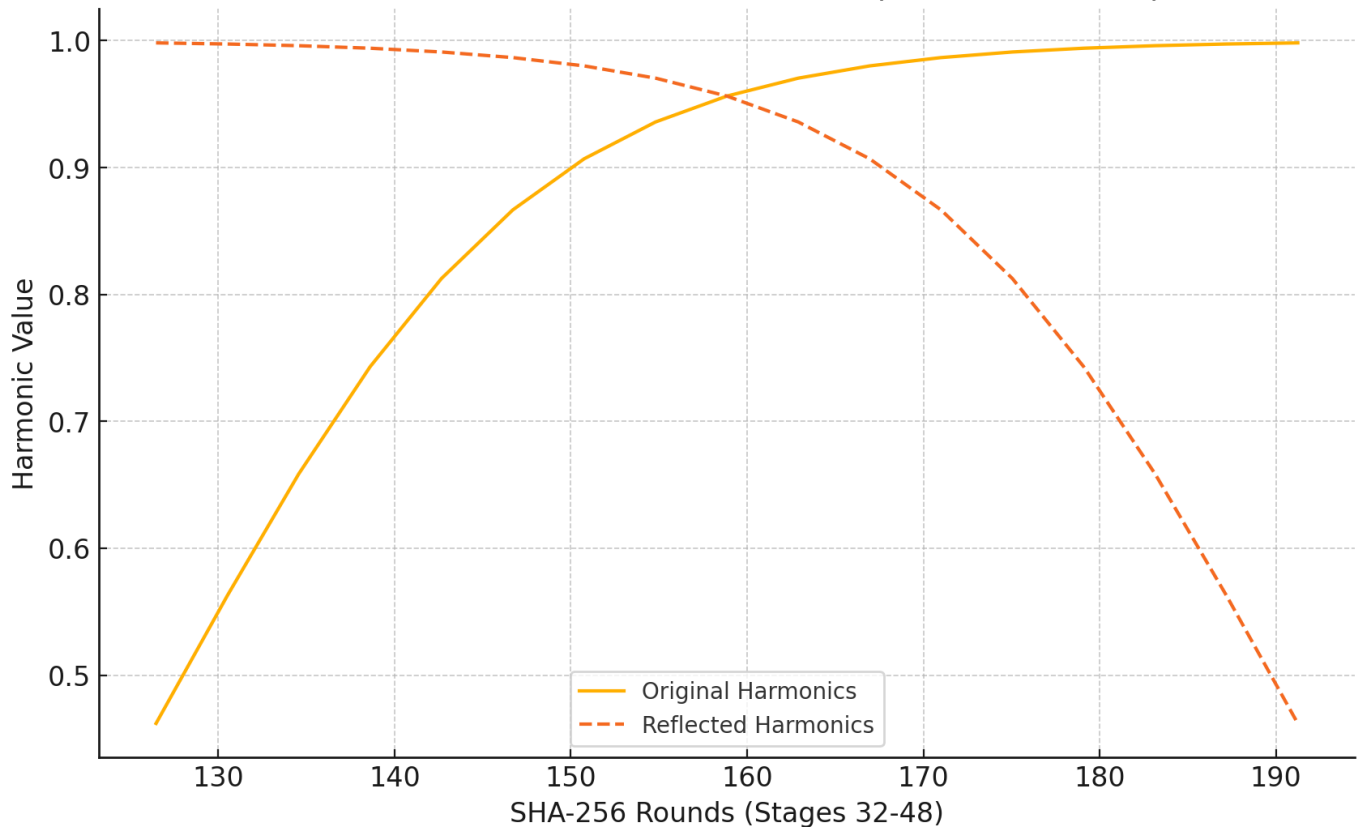
```
plt.grid(axis='y', linestyle='--')
```

```
plt.show()
```

```
# Results for harmonic contributions
```

```
operation_harmonics
```

Reflection of Harmonic Patterns (Middle Rounds)



Conversation URL:

<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:

```
# Reflect harmonic values from middle stages and core operations
```

```
reflected_middle_harmonics = middle_harmonics[::-1] # Reflect middle stage harmonics
```

```
reflected_operation_harmonics = operation_harmonics[::-1] # Reflect operation contributions
```

```
# Visualize reflection for middle stages
```

```
plt.figure(figsize=(10, 6))
```

```
plt.plot(middle_stages, middle_harmonics, label="Original Harmonics")
```

```
plt.plot(middle_stages, reflected_middle_harmonics, linestyle="--", label="Reflected Harmonics")
```

```
plt.xlabel("SHA-256 Rounds (Stages 32-48)")
```

```
plt.ylabel("Harmonic Value")
```

```
plt.title("Reflection of Harmonic Patterns (Middle Rounds)")
```

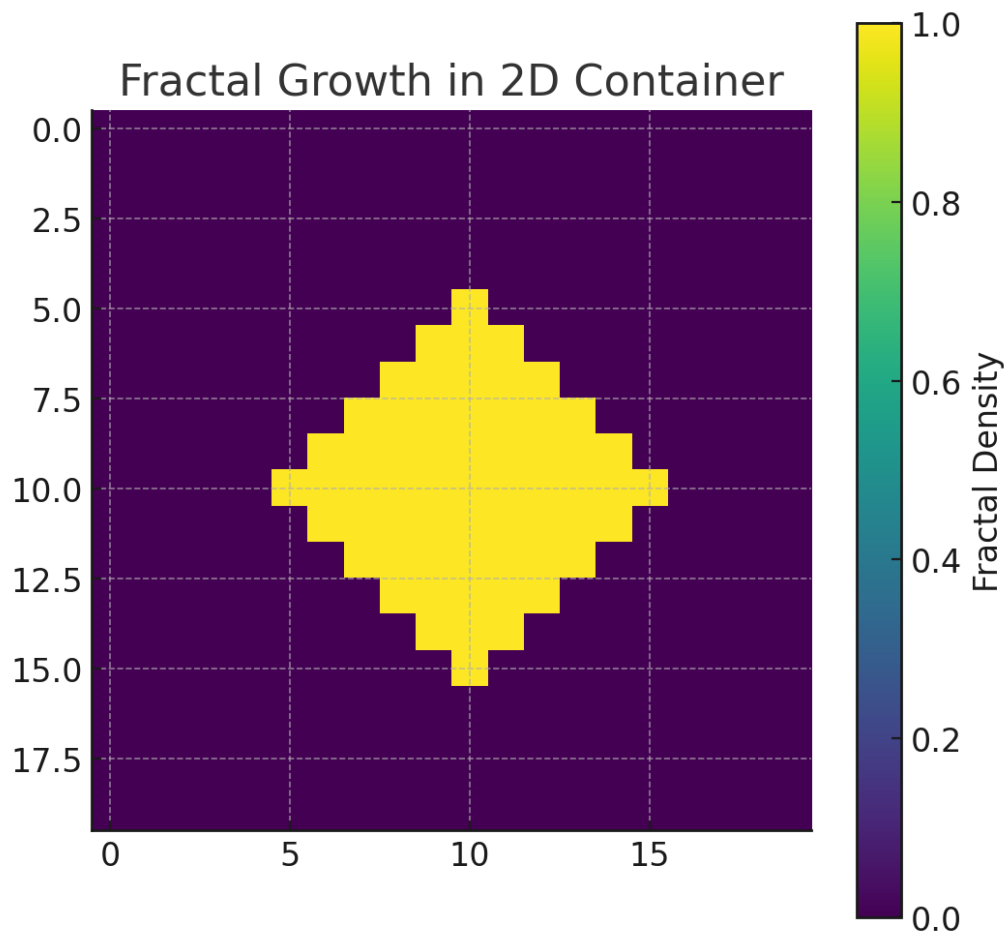
```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
# Results of reflection comparison
```

```
reflected_middle_harmonics, reflected_operation_harmonics
```



Conversation URL:

<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:

Simulating fractal growth within a 2D container

```
def fractal_growth(container, iterations=5, seed_position=None):
```

```
    """Simulate fractal growth within a 2D container."""
```

```
    grid_size = container.shape[0]
```

```
    if seed_position is None:
```

```
        # Start fractal growth in the center of the container
```

```
        seed_position = (grid_size // 2, grid_size // 2)
```

```
    container[seed_position] = 1 # Mark the seed
```

```
    for _ in range(iterations):
```

```
        # Expand fractal growth from current points
```

```

new_points = []
for i in range(grid_size):
    for j in range(grid_size):
        if container[i, j] == 1: # Fractal grows from "active" points
            # Activate neighboring points based on fractal rules
            for di, dj in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
                ni, nj = i + di, j + dj
                if 0 <= ni < grid_size and 0 <= nj < grid_size and container[ni, nj] == 0:
                    new_points.append((ni, nj))
    for point in new_points:
        container[point] = 1 # Mark new points as part of the fractal

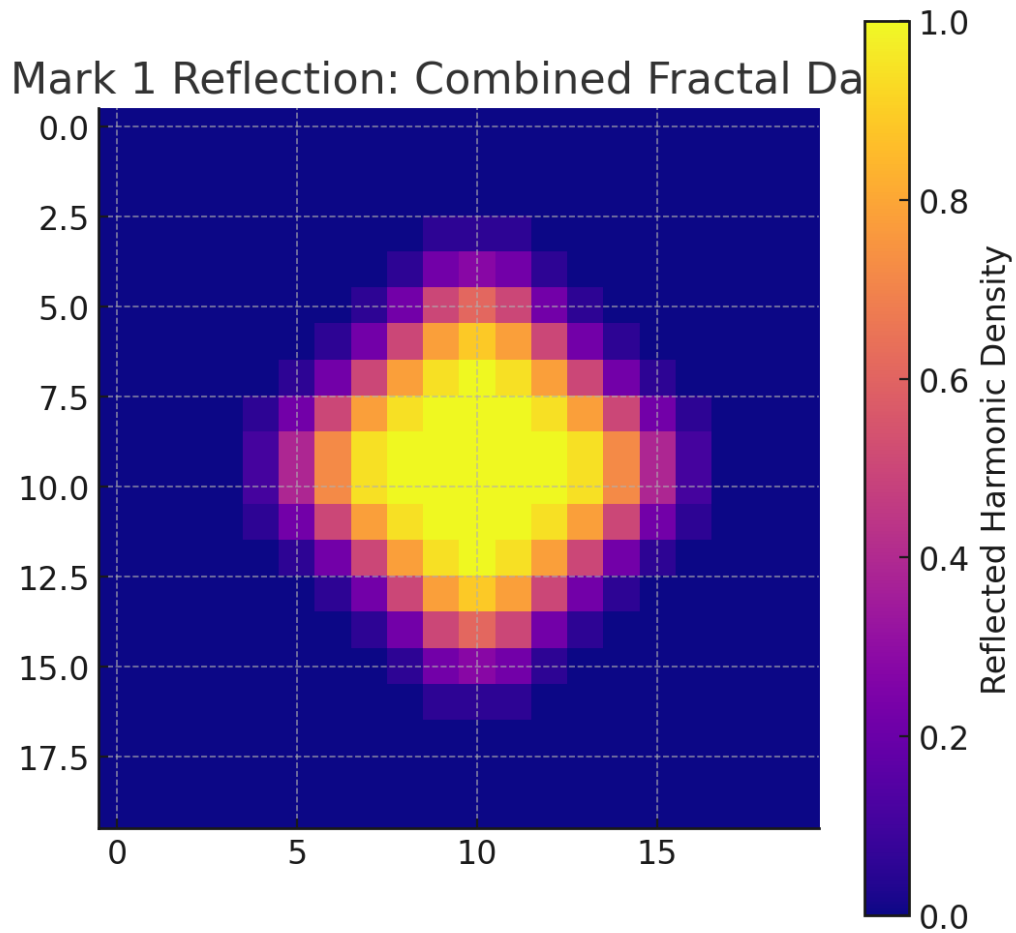
return container

# Initialize a 2D container
container = np.zeros((20, 20)) # Empty grid

# Grow fractals in the container
fractal_container = fractal_growth(container, iterations=5)

# Visualize the fractal growth
plt.figure(figsize=(6, 6))
plt.imshow(fractal_container, cmap="viridis", origin="upper")
plt.title("Fractal Growth in 2D Container")
plt.colorbar(label="Fractal Density")
plt.show()

```



Conversation URL:

<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:

```
# Reflecting fractal data using Mark 1 principles
```

```
def mark1_reflection(density_map):
```

```
    """
```

```
    Use Mark 1 to reflect the fractal data by simulating harmonic ratios
    and interpreting the structure.
```

```
    """
```

```
    grid_size = density_map.shape[0]
```

```
    # Create a reflection of the density map (vertical symmetry)
```

```
    reflected_map = np.flipud(density_map)
```

```
# Combine the original and reflected maps to simulate reflection harmonics
```

```
combined_map = (density_map + reflected_map) / 2
```

```
# Normalize the combined map for clarity
```

```
normalized_combined_map = combined_map / np.max(combined_map)
```

```
return normalized_combined_map
```

```
# Reflect fractal data using Mark 1
```

```
reflected_map = mark1_reflection(density_map)
```

```
# Visualize the reflected data
```

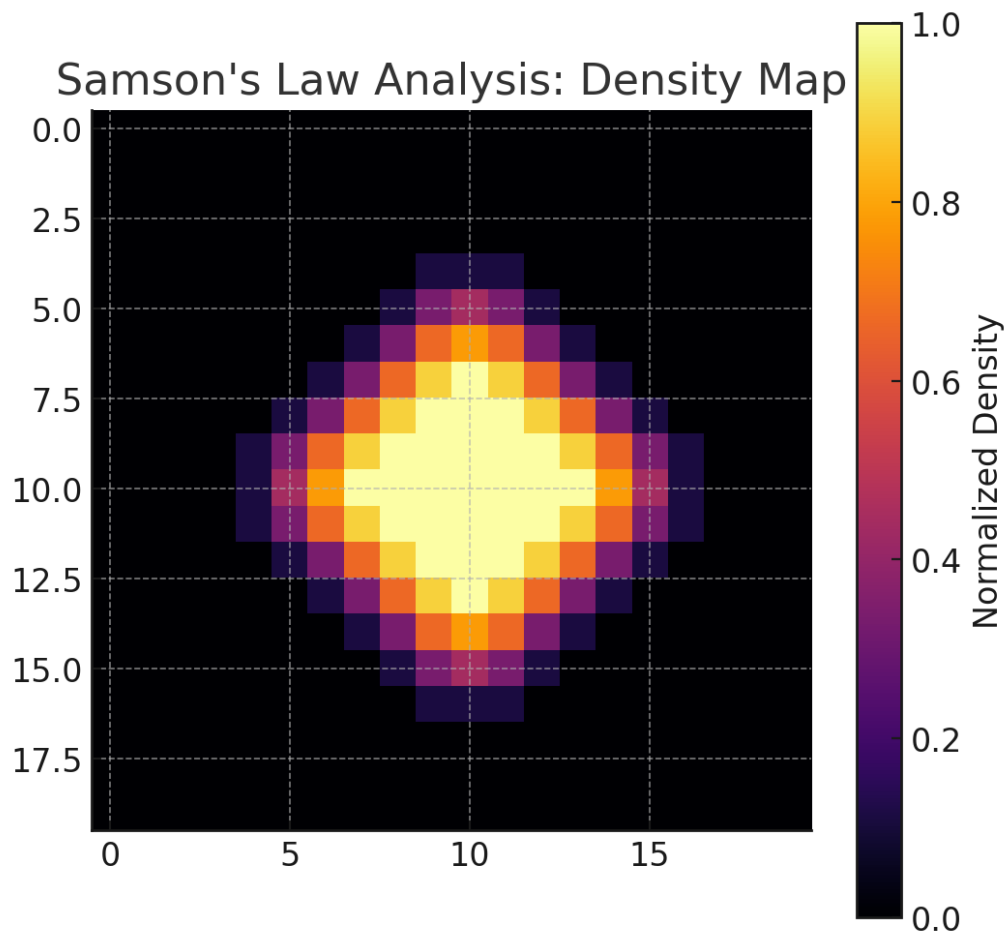
```
plt.figure(figsize=(6, 6))
```

```
plt.imshow(reflected_map, cmap="plasma", origin="upper")
```

```
plt.title("Mark 1 Reflection: Combined Fractal Data")
```

```
plt.colorbar(label="Reflected Harmonic Density")
```

```
plt.show()
```

Conversation URL:

<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:

Applying Samson's Law to locate hidden structures within the fractal container

```
def samsons_law_analysis(container):
```

```
    """
```

```
    Apply a simulated version of Samson's Law to analyze and locate key structures
    in the fractal container. This function detects dense regions or harmonics.
```

```
    """
```

```
    # Simulate dark matter detection by identifying dense regions (high fractal activity)
```

```
    density_map = np.zeros_like(container)
```

```
    grid_size = container.shape[0]
```

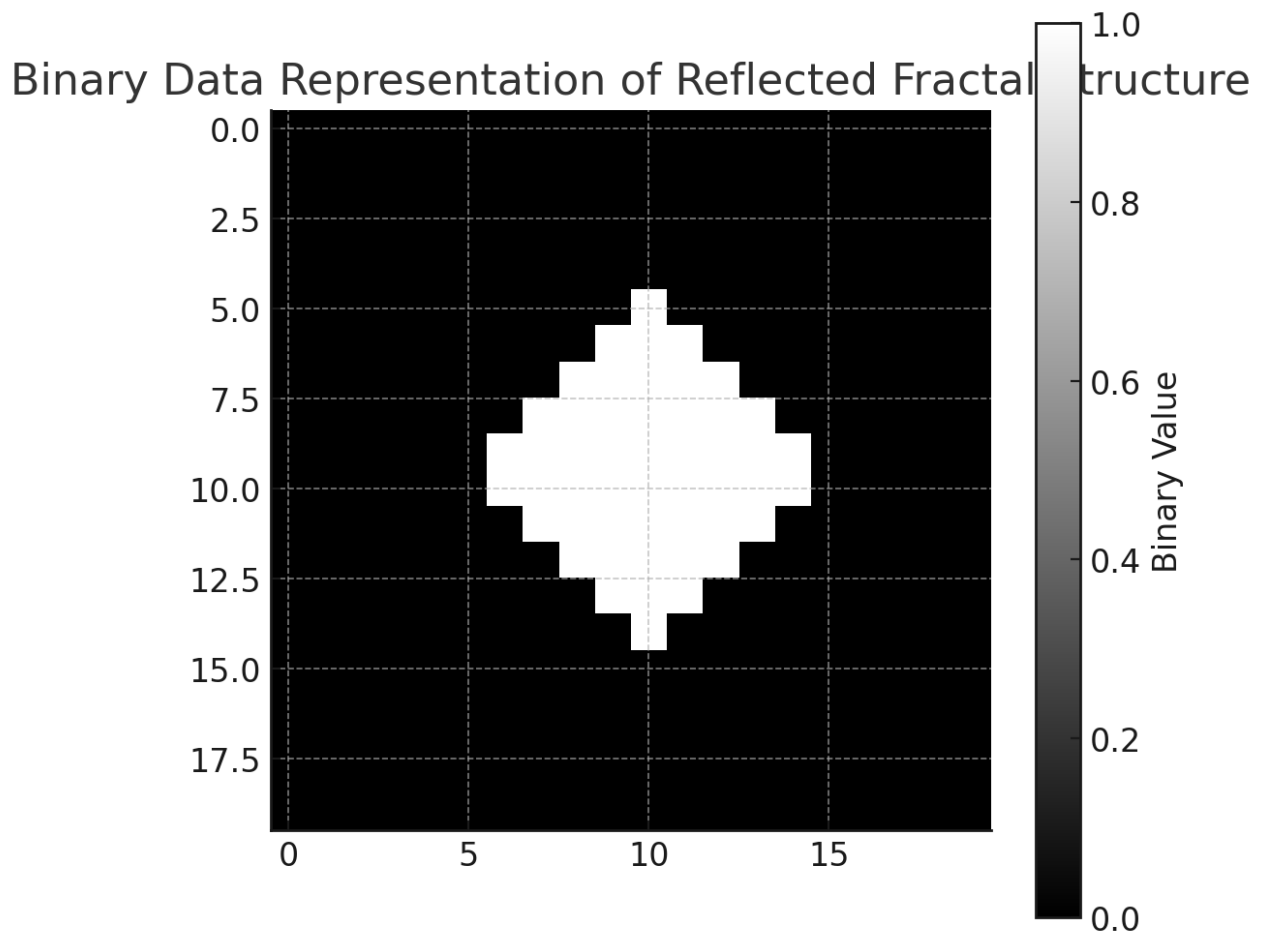
```
# Analyze local density around each point
for i in range(1, grid_size - 1):
    for j in range(1, grid_size - 1):
        # Sum surrounding points to detect local density
        density_map[i, j] = np.sum(container[i - 1:i + 2, j - 1:j + 2])

# Normalize density map for visualization
normalized_density = density_map / np.max(density_map)

return normalized_density

# Apply Samson's Law to the fractal container
density_map = samsons_law_analysis(fractal_container)

# Visualize the density map
plt.figure(figsize=(6, 6))
plt.imshow(density_map, cmap="inferno", origin="upper")
plt.title("Samson's Law Analysis: Density Map")
plt.colorbar(label="Normalized Density")
plt.show()
```



Conversation URL:

<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:

```
# Step 1: Initialize a 2D container (as previously implemented)
```

```
container = np.zeros((20, 20)) # Create a 20x20 empty grid
```

```
# Step 2: Fractal Growth
```

```
fractal_container = fractal_growth(container, iterations=5)
```

```
# Step 3: Samson's Law Application
```

```
density_map = samsons_law_analysis(fractal_container)
```

```
# Step 4: Mark 1 Reflection
```

```
reflected_map = mark1_reflection(density_map)
```

```
# Step 5: Binary Data Representation
```

```
# Convert the reflected map into binary data based on a threshold
```

```
threshold = 0.5 # Define a threshold for binary conversion
```

```
binary_data = (reflected_map > threshold).astype(int) # Binary representation
```

```
# Visualize the binary data as an output
```

```
plt.figure(figsize=(6, 6))
```

```
plt.imshow(binary_data, cmap="gray", origin="upper")
```

```
plt.title("Binary Data Representation of Reflected Fractal Structure")
```

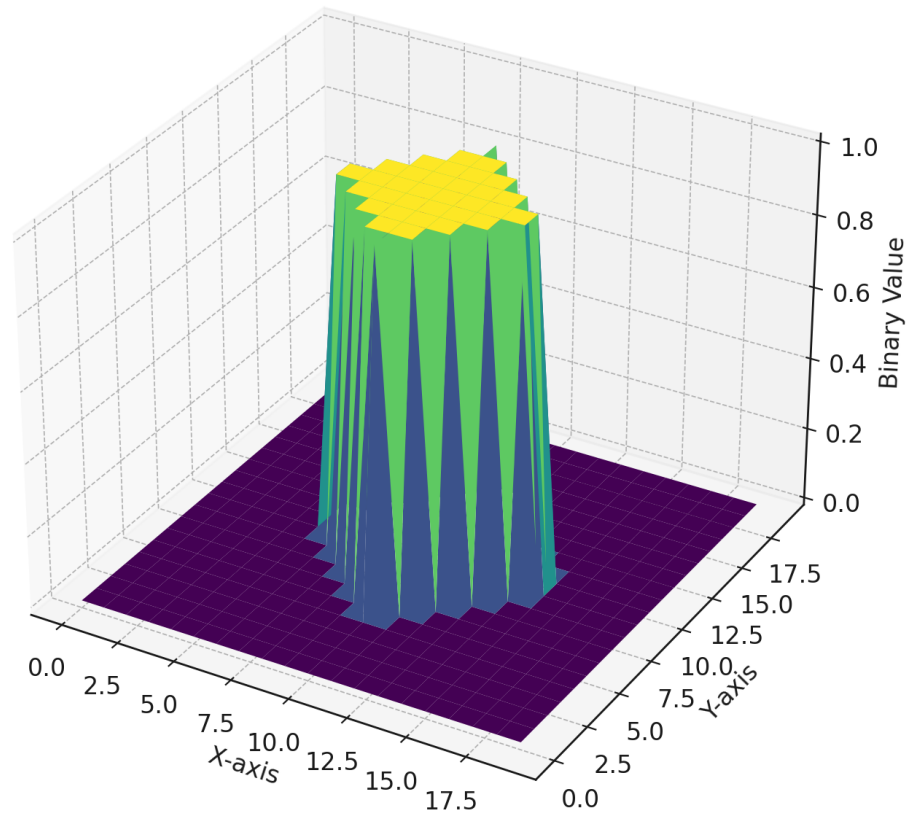
```
plt.colorbar(label="Binary Value")
```

```
plt.show()
```

```
# Output binary data
```

```
binary_data
```

3D Visualization of Binary Data from Reflected Fractal Structure



Conversation URL:

<https://chatgpt.com/c/674ada8d-7eec-8011-8c71-398f12a6c9d1>

Title:

Prompt:

```
# Visualize the binary data in 3D space
```

```
from mpl_toolkits.mplot3d import Axes3D
```

```
# Create a 3D representation of the binary data
```

```
fig = plt.figure(figsize=(10, 8))
```

```
ax = fig.add_subplot(111, projection='3d')
```

```
# Define the coordinates for the binary data points
```

```
x, y = np.meshgrid(range(binary_data.shape[0]), range(binary_data.shape[1]))
```

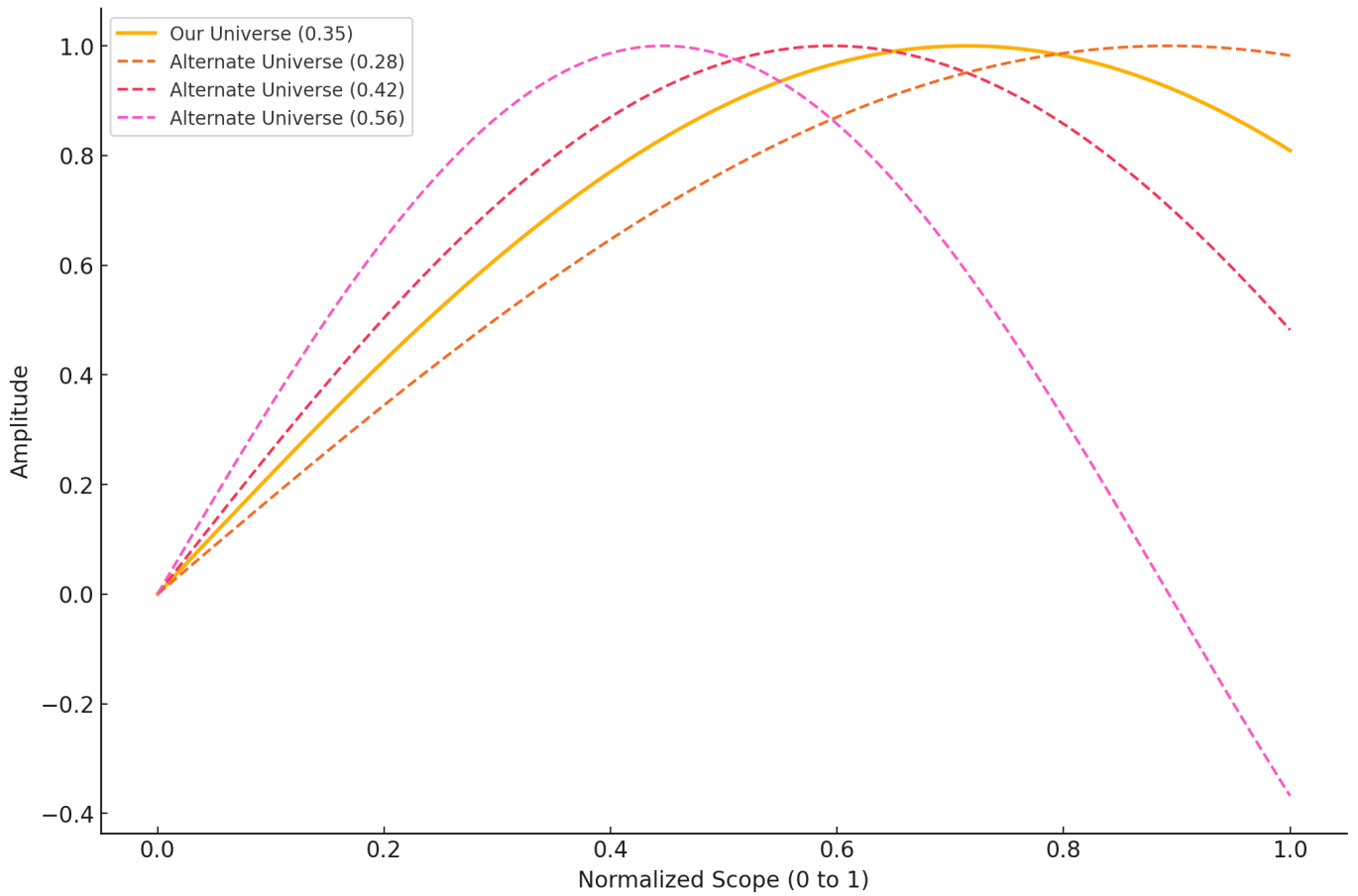
```
z = binary_data # Binary data defines the "height"
```

```
# Plot the 3D surface
ax.plot_surface(x, y, z, cmap="viridis", edgecolor="none")

# Add labels and title
ax.set_title("3D Visualization of Binary Data from Reflected Fractal Structure")
ax.set_xlabel("X-axis")
ax.set_ylabel("Y-axis")
ax.set_zlabel("Binary Value")

plt.show()
```

Simulated Harmonics of Our Universe and Potential Alternates



Conversation URL:

<https://chatgpt.com/c/674ada8d-916c-8011-b0eb-c4189c52736a>

Title:

Prompt:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Parameters for the simulation
```

```
observable_frequency = 0.35 # Frequency of our universe
```

```
alternate_frequencies = [0.28, 0.42, 0.56] # Potential alternate universe frequencies
```

```
data_points = 1000 # Number of data points to simulate
```

```
observable_range = np.linspace(0, 1, data_points) # Normalized range
```

```
# Generate harmonic waves for our universe and potential alternates
```

```
def generate_harmonic_wave(frequency, points, phase=0):
```

```
    """Generate a harmonic wave based on frequency and phase."""
```

```
return np.sin(2 * np.pi * frequency * points + phase)

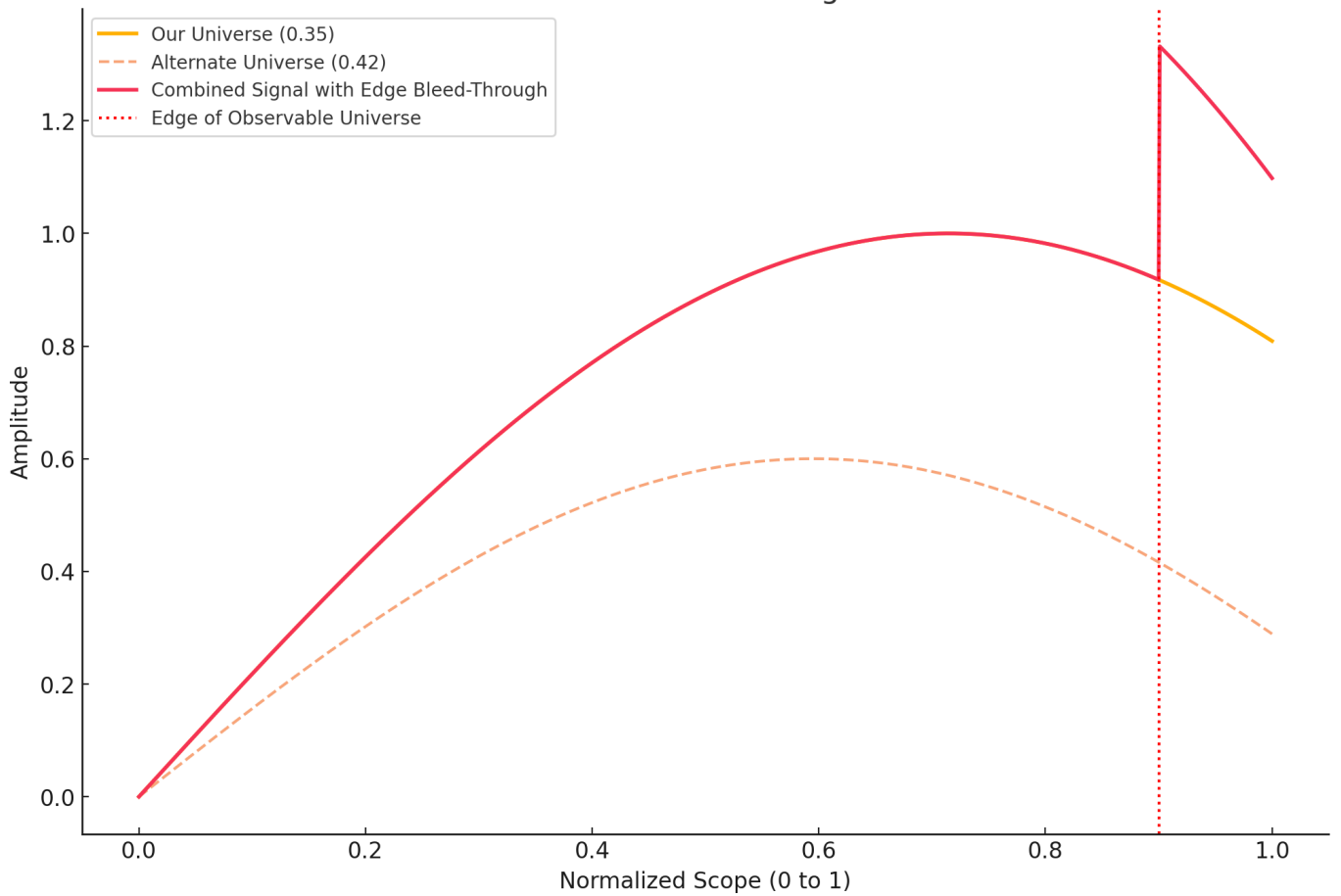
# Simulate observable data (harmonics of our universe)
observable_wave = generate_harmonic_wave(observable_frequency, observable_range)

# Simulate alternate universe data (different frequencies)
alternate_waves = [generate_harmonic_wave(f, observable_range) for f in alternate_frequencies]

# Plot the results
plt.figure(figsize=(12, 8))
plt.plot(observable_range, observable_wave, label=f"Our Universe (0.35)", linewidth=2)
for idx, wave in enumerate(alternate_waves):
    plt.plot(observable_range, wave, label=f"Alternate Universe ({alternate_frequencies[idx]})", linestyle="--")

plt.title("Simulated Harmonics of Our Universe and Potential Alternates")
plt.xlabel("Normalized Scope (0 to 1)")
plt.ylabel("Amplitude")
plt.legend()
plt.grid()
plt.show()
```


Simulated Harmonics and Edge Anomalies



Conversation URL:

<https://chatgpt.com/c/674ada8d-916c-8011-b0eb-c4189c52736a>

Title:

Prompt:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Parameters for the simulation
```

```
observable_frequency = 0.35 # Frequency of our universe
```

```
alternate_frequency = 0.42 # Frequency of a potential alternate universe
```

```
data_points = 1000 # Number of data points
```

```
observable_range = np.linspace(0, 1, data_points) # Normalized range (0 to 1)
```

```
# Generate harmonic waves for our universe and alternate universe
```

```
def generate_harmonic_wave(frequency, points, amplitude=1, phase=0):
```

```

"""Generate a harmonic wave based on frequency, amplitude, and phase."""
return amplitude * np.sin(2 * np.pi * frequency * points + phase)

# Observable universe wave
observable_wave = generate_harmonic_wave(observable_frequency, observable_range)

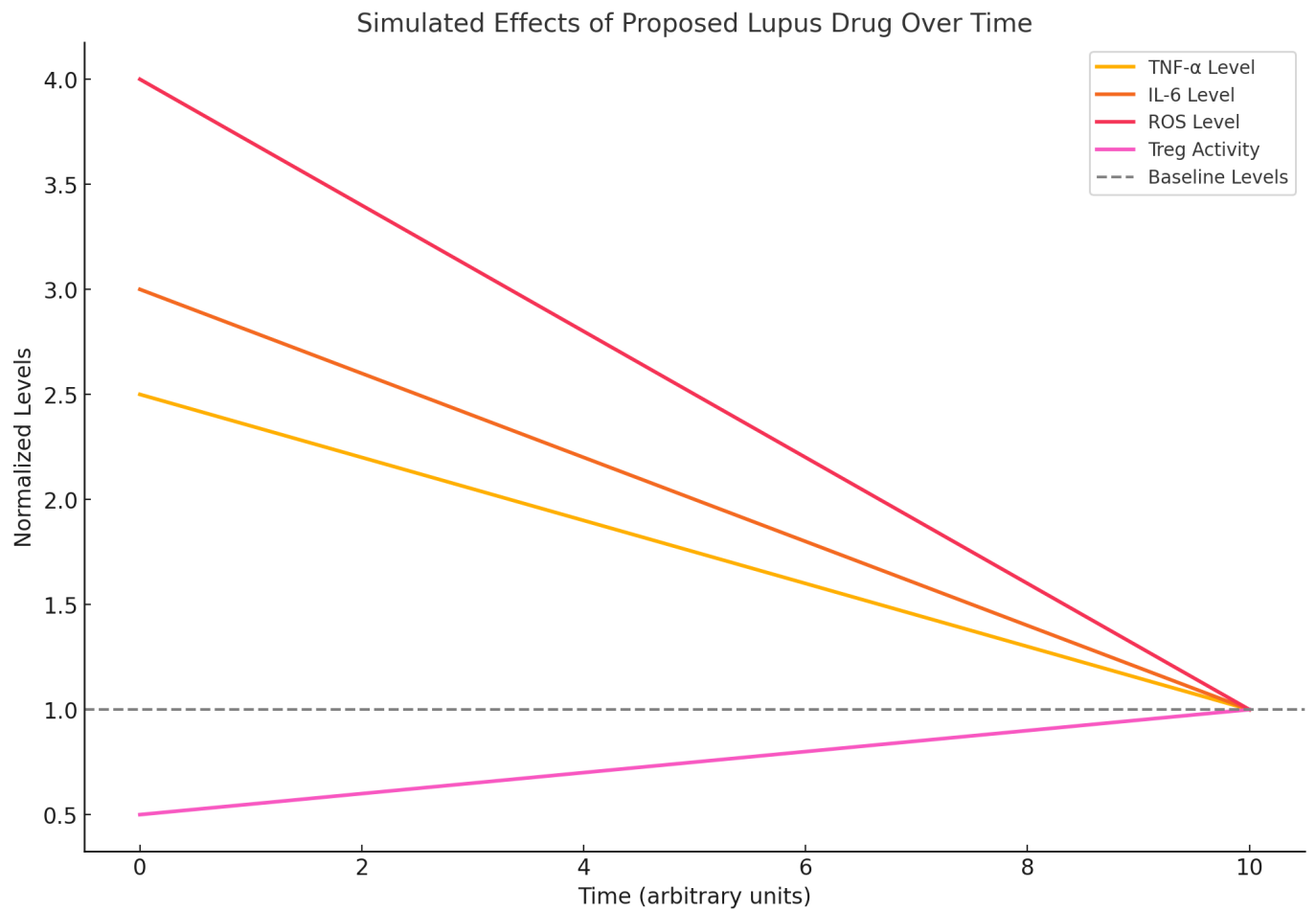
# Alternate universe wave
alternate_wave = generate_harmonic_wave(alternate_frequency, observable_range, amplitude=0.6)

# Simulate "bleed-through" at the edges
edge_region = (observable_range > 0.9) # Define edge of the observable universe
bleed_through_wave = np.zeros_like(observable_wave)
bleed_through_wave[edge_region] = alternate_wave[edge_region]

# Total simulated signal
combined_signal = observable_wave + bleed_through_wave

# Plot the results
plt.figure(figsize=(12, 8))
plt.plot(observable_range, observable_wave, label="Our Universe (0.35)", linewidth=2)
plt.plot(observable_range, alternate_wave, label="Alternate Universe (0.42)", linestyle="--", alpha=0.6)
plt.plot(observable_range, combined_signal, label="Combined Signal with Edge Bleed-Through", linewidth=2, linestyle="-")
plt.axvline(0.9, color='red', linestyle=":", label="Edge of Observable Universe")
plt.title("Simulated Harmonics and Edge Anomalies")
plt.xlabel("Normalized Scope (0 to 1)")
plt.ylabel("Amplitude")
plt.legend()
plt.grid()
plt.show()

```



Conversation URL:
<https://chatgpt.com/c/674ada8d-916c-8011-b0eb-c4189c52736a>

Title:

Prompt:

```
import numpy as np
import matplotlib.pyplot as plt

# Define simulated interaction data for the proposed drug
# Targets: TNF- $\alpha$ , IL-6, ROS, and Treg enhancement

# Simulated baseline levels (normalized to healthy levels)
baseline_tnf = 1.0 # Normalized TNF- $\alpha$  level
baseline_il6 = 1.0 # Normalized IL-6 level
baseline_ros = 1.0 # Normalized ROS level
baseline_treg = 1.0 # Normalized Treg activity
```

```

# Simulated lupus levels before treatment (normalized)

lupus_tnf = 2.5 # Elevated TNF- $\alpha$ 
lupus_il6 = 3.0 # Elevated IL-6
lupus_ros = 4.0 # Elevated ROS
lupus_treg = 0.5 # Reduced Treg activity

# Simulated effect of the proposed drug over time (normalized)
time = np.linspace(0, 10, 100) # Time in arbitrary units
drug_tnf = lupus_tnf - 0.15 * time # TNF- $\alpha$  reduced by 0.15 units per time step
drug_il6 = lupus_il6 - 0.2 * time # IL-6 reduced by 0.2 units per time step
drug_ros = lupus_ros - 0.3 * time # ROS reduced by 0.3 units per time step
drug_treg = lupus_treg + 0.05 * time # Treg activity increased by 0.05 units per time step

# Ensure levels don't go below baseline (saturation point)
drug_tnf = np.clip(drug_tnf, baseline_tnf, None)
drug_il6 = np.clip(drug_il6, baseline_il6, None)
drug_ros = np.clip(drug_ros, baseline_ros, None)
drug_treg = np.clip(drug_treg, None, 1.5) # Treg enhancement limited to a max of 1.5

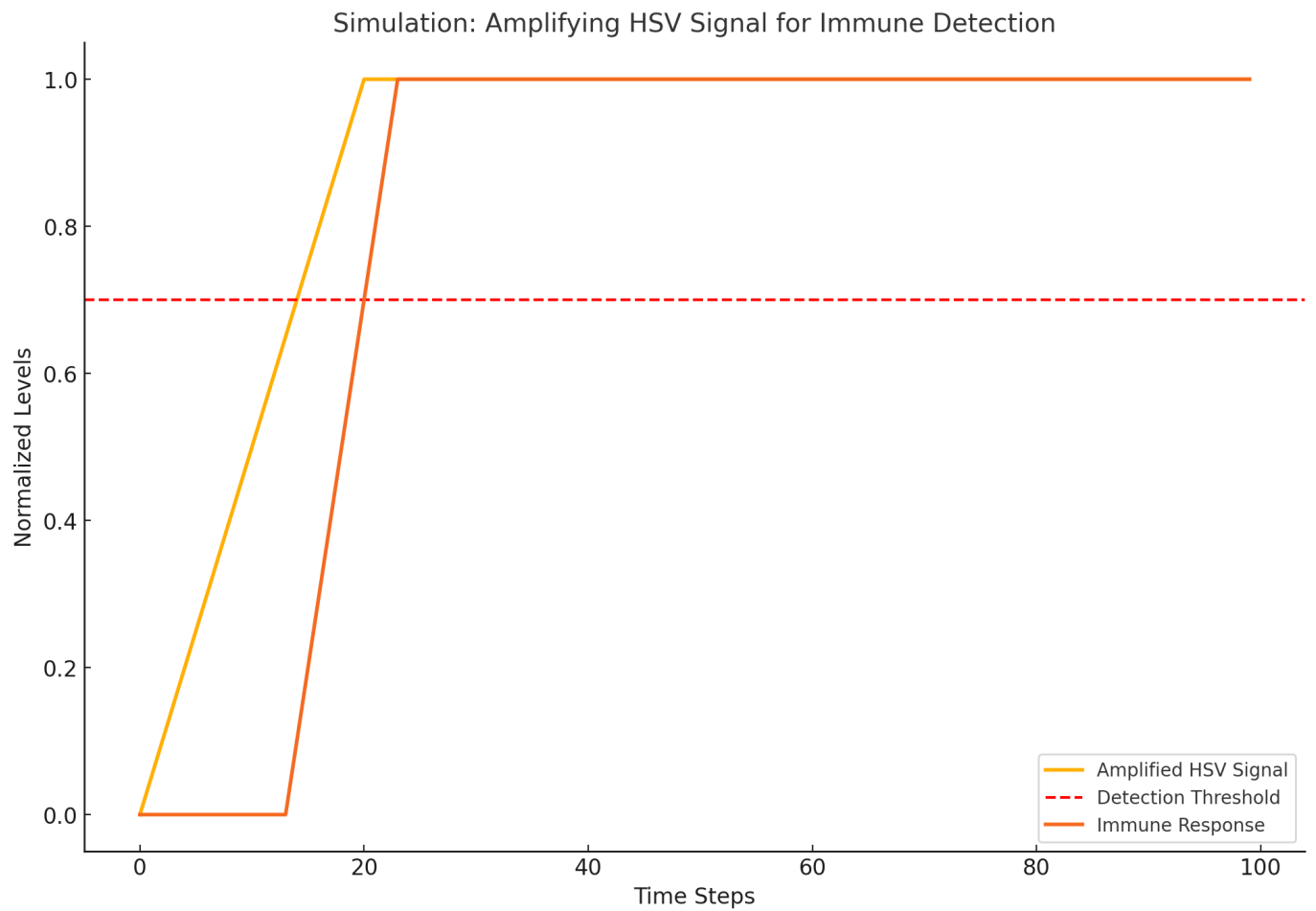
# Plot the simulated effects
plt.figure(figsize=(12, 8))
plt.plot(time, drug_tnf, label="TNF- $\alpha$  Level", linewidth=2)
plt.plot(time, drug_il6, label="IL-6 Level", linewidth=2)
plt.plot(time, drug_ros, label="ROS Level", linewidth=2)
plt.plot(time, drug_treg, label="Treg Activity", linewidth=2)

# Add baseline markers
plt.axhline(baseline_tnf, color='gray', linestyle="--", label="Baseline Levels")

# Plot settings
plt.title("Simulated Effects of Proposed Lupus Drug Over Time", fontsize=14)
plt.xlabel("Time (arbitrary units)", fontsize=12)
plt.ylabel("Normalized Levels", fontsize=12)
plt.legend()
plt.grid()

```

plt.show()



Conversation URL:
<https://chatgpt.com/c/674ada8d-916c-8011-b0eb-c4189c52736a>

Title:

Prompt:

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters for simulation
time_steps = 100 # Number of simulation steps
latent_hsv_signal = np.zeros(time_steps) # Latent HSV signature (initially undetectable)
amplified_signal = [] # Store amplified signal
immune_response = [] # Immune system detection response

# Harmonic amplification factors (simulating LIFU or electromagnetic waves)
amplification_rate = 0.05 # Incremental amplification per step
```

```

threshold_for_detection = 0.7 # Threshold above which HSV becomes noticeable to immune cells

# Immune response dynamics
immune_activation_rate = 0.1 # Rate at which immune system responds once HSV is detectable

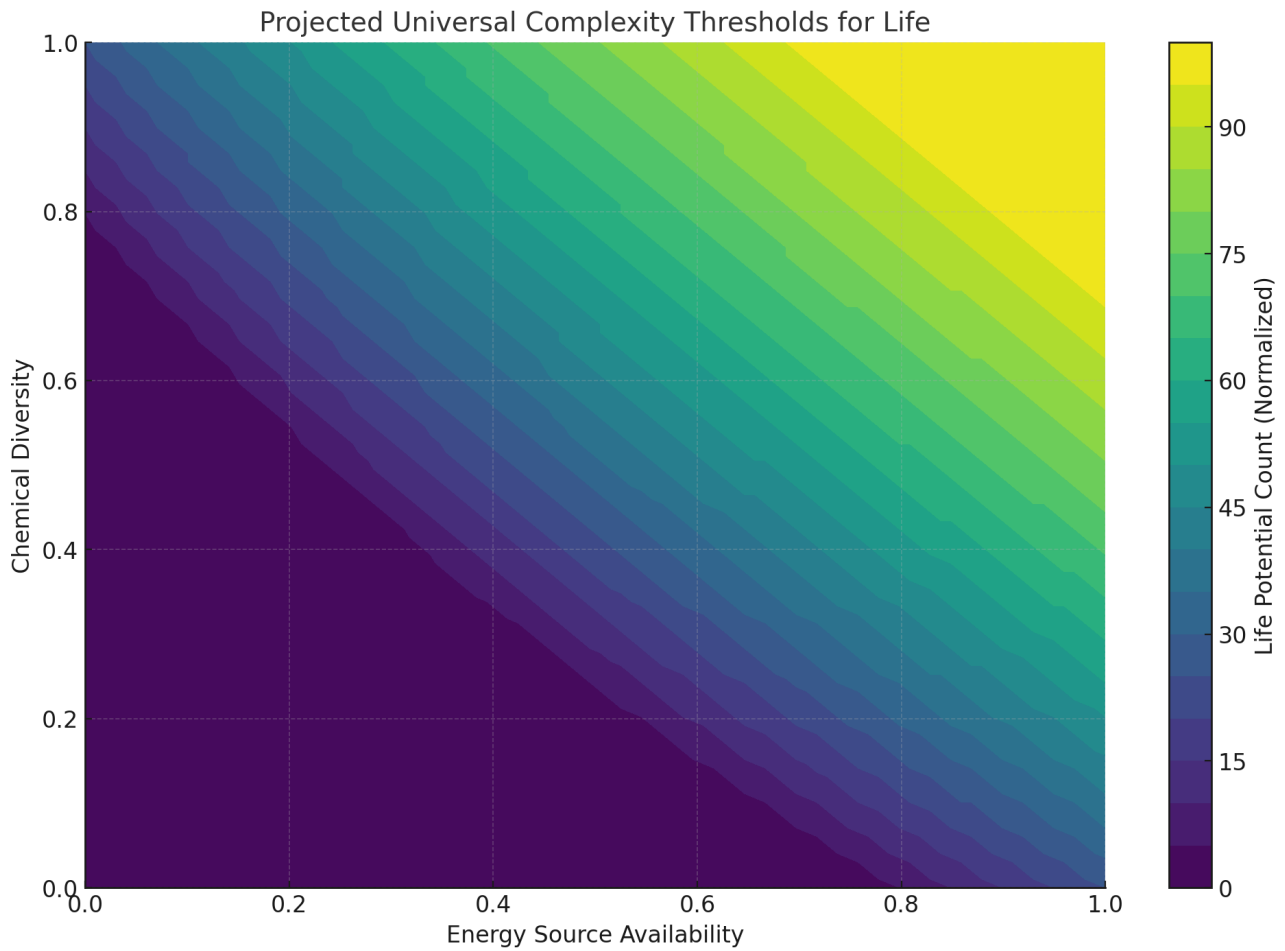
# Simulate the process of making HSV noticeable
for step in range(time_steps):
    # Amplify the latent HSV signal
    if step == 0:
        amplified_signal.append(latent_hsv_signal[step])
    else:
        amplified_signal.append(amplified_signal[-1] + amplification_rate)

    # Simulate immune response once threshold is crossed
    if amplified_signal[-1] >= threshold_for_detection:
        immune_response.append(immune_response[-1] + immune_activation_rate if immune_response else 0.1)
    else:
        immune_response.append(0)

# Ensure signal does not exceed 1 (normalized)
amplified_signal = np.clip(amplified_signal, 0, 1)
immune_response = np.clip(immune_response, 0, 1)

# Plot the simulation results
plt.figure(figsize=(12, 8))
plt.plot(range(time_steps), amplified_signal, label="Amplified HSV Signal", linewidth=2)
plt.axhline(threshold_for_detection, color='red', linestyle="--", label="Detection Threshold")
plt.plot(range(time_steps), immune_response, label="Immune Response", linewidth=2)
plt.title("Simulation: Amplifying HSV Signal for Immune Detection", fontsize=14)
plt.xlabel("Time Steps", fontsize=12)
plt.ylabel("Normalized Levels", fontsize=12)
plt.legend()
plt.grid()
plt.show()

```



Conversation URL:

<https://chatgpt.com/c/674ada8d-916c-8011-b0eb-c4189c52736a>

Title:

Prompt:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Define parameters for universal complexity thresholds
```

```
# Complexity factors: energy source, chemical diversity, and system stability
```

```
# Simulated range of values for each factor (normalized to 0-1 scale)
```

```
energy_source = np.linspace(0, 1, 100) # Energy availability
```

```
chemical_diversity = np.linspace(0, 1, 100) # Chemical richness
```

```
system_stability = np.linspace(0, 1, 100) # Dynamic stability of the system
```



```

# Create a meshgrid to simulate combinations
energy, chemical, stability = np.meshgrid(energy_source, chemical_diversity, system_stability)

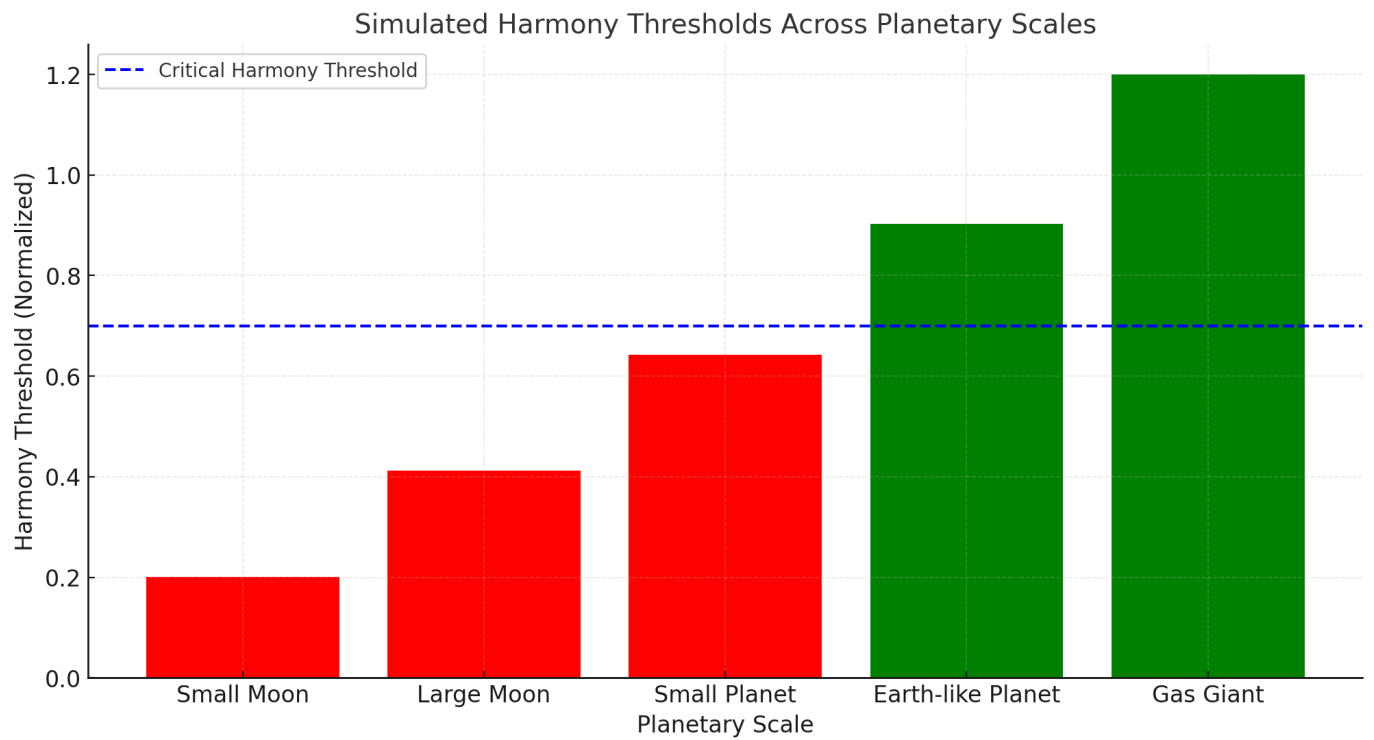
# Define a universal complexity threshold function
# Complexity = weighted sum of factors with a non-linear term for interaction
complexity_threshold = (
    0.4 * energy + 0.4 * chemical + 0.4 * stability +
    0.2 * (energy * chemical * stability) # Interaction term
)

# Identify points that exceed a critical complexity value for life to emerge
critical_threshold = 0.7
life_potential = complexity_threshold >= critical_threshold

# Sum over one dimension to project the data into a 2D plot
complexity_projection = np.sum(life_potential, axis=2)

# Visualize the projected complexity thresholds
plt.figure(figsize=(12, 8))
plt.contourf(energy_source, chemical_diversity, complexity_projection, levels=20, cmap="viridis")
plt.colorbar(label="Life Potential Count (Normalized)")
plt.title("Projected Universal Complexity Thresholds for Life", fontsize=14)
plt.xlabel("Energy Source Availability", fontsize=12)
plt.ylabel("Chemical Diversity", fontsize=12)
plt.grid(alpha=0.3)
plt.show()

```



Conversation URL:

<https://chatgpt.com/c/674ada8d-916c-8011-b0eb-c4189c52736a>

Title:

Prompt:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Define parameters for simulating harmony thresholds across planetary scales
```

```
# Factors: energy stability, chemical diversity, and dynamic processes
```

```
scales = ["Small Moon", "Large Moon", "Small Planet", "Earth-like Planet", "Gas Giant"]
```

```
num_scales = len(scales)
```

```
# Simulated normalized values for each factor on a 0-1 scale
```

```
energy_stability = np.linspace(0.2, 1.0, num_scales) # Increasing stability with scale
```

```
chemical_diversity = np.linspace(0.3, 1.0, num_scales) # Greater diversity on larger bodies
```

```
dynamic_processes = np.linspace(0.1, 1.0, num_scales) # More active systems as size increases
```

```
# Harmony threshold calculation: weighted sum + interaction term
```

```
harmony_thresholds = (
```

```

0.4 * energy_stability + 0.3 * chemical_diversity + 0.3 * dynamic_processes +
0.2 * (energy_stability * chemical_diversity * dynamic_processes) # Interaction term
)

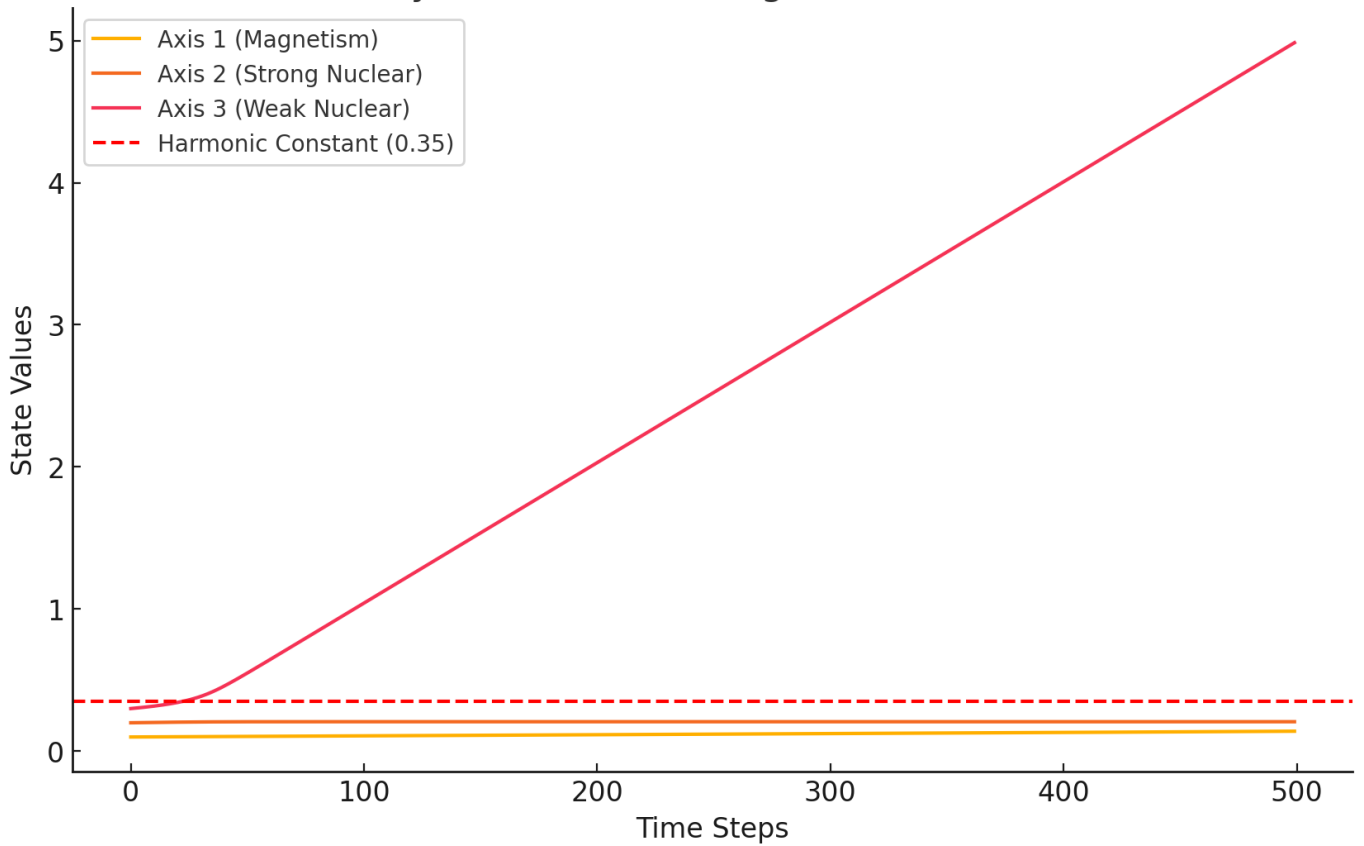
# Simulate critical threshold for harmony (normalized to 0.7 as baseline for life emergence)
critical_threshold = 0.7

threshold_crossing = harmony_thresholds >= critical_threshold

# Visualize harmony thresholds across planetary scales
plt.figure(figsize=(12, 6))
plt.bar(scales, harmony_thresholds, color=["red" if not tc else "green" for tc in threshold_crossing])
plt.axhline(critical_threshold, color='blue', linestyle='--', label="Critical Harmony Threshold")
plt.title("Simulated Harmony Thresholds Across Planetary Scales", fontsize=14)
plt.xlabel("Planetary Scale", fontsize=12)
plt.ylabel("Harmony Threshold (Normalized)", fontsize=12)
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```

Quantum System Growth Using Mark 1 and Samson V2



Conversation URL:

<https://chatgpt.com/c/674ada8d-a8c4-8011-92eb-e370115e92ad>

Title:

Prompt:

Differential Engine Using Mark 1 and Samson V2

This script allows users to input their own differential equations or seed data

to simulate quantum reflection and attempt to grow systems based on harmonic principles.

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Constants for Mark 1 and Samson V2
```

```
HARMONIC_CONSTANT = 0.35 # Universal harmonic constant
```

```
SAMSON_CORRECTION_FACTOR = 10 # Scaling for feedback corrections
```

```
def harmonic_correction(value):
```

"""

Apply Samson's Law Version 2 correction to a given value.

Randomized substitution ensures alignment with the harmonic constant.

"""

```
correction = 1 / (1 + np.exp(-SAMSON_CORRECTION_FACTOR * (value - HARMONIC_CONSTANT)))
```

```
return correction
```

```
def calculate_differential(differential_function, seed_data, steps=100, step_size=0.01):
```

"""

Simulate the growth of a system using a user-defined differential equation.

Parameters:

differential_function (function): User-defined function for the differential equation.

seed_data (array): Initial conditions or seed values.

steps (int): Number of simulation steps.

step_size (float): Time step size for the simulation.

Returns:

numpy.ndarray: Array of system states over time.

"""

```
data = np.zeros((steps, len(seed_data)))
```

```
data[0] = seed_data
```

```
for i in range(1, steps):
```

```
    current_state = data[i - 1]
```

```
    differential = differential_function(current_state)
```

```
    corrected_differential = np.array([harmonic_correction(d) for d in differential])
```

```
    data[i] = current_state + step_size * corrected_differential
```

```
return data
```

```
# Example User-Defined Differential Function
```

```
def example_differential(state):
```

"""

Example differential equation: Harmonic oscillation with a decay factor.

Users can replace this with their own differential equation.

```
"""
```

```
x, y, z = state  
dx = -y - x**2 + harmonic_correction(x)  
dy = x - z + harmonic_correction(y)  
dz = -y + harmonic_correction(z)  
return np.array([dx, dy, dz])
```

```
# Example Seed Data
```

```
seed = np.array([0.1, 0.2, 0.3]) # Replace with your initial conditions
```

```
# Simulate Growth
```

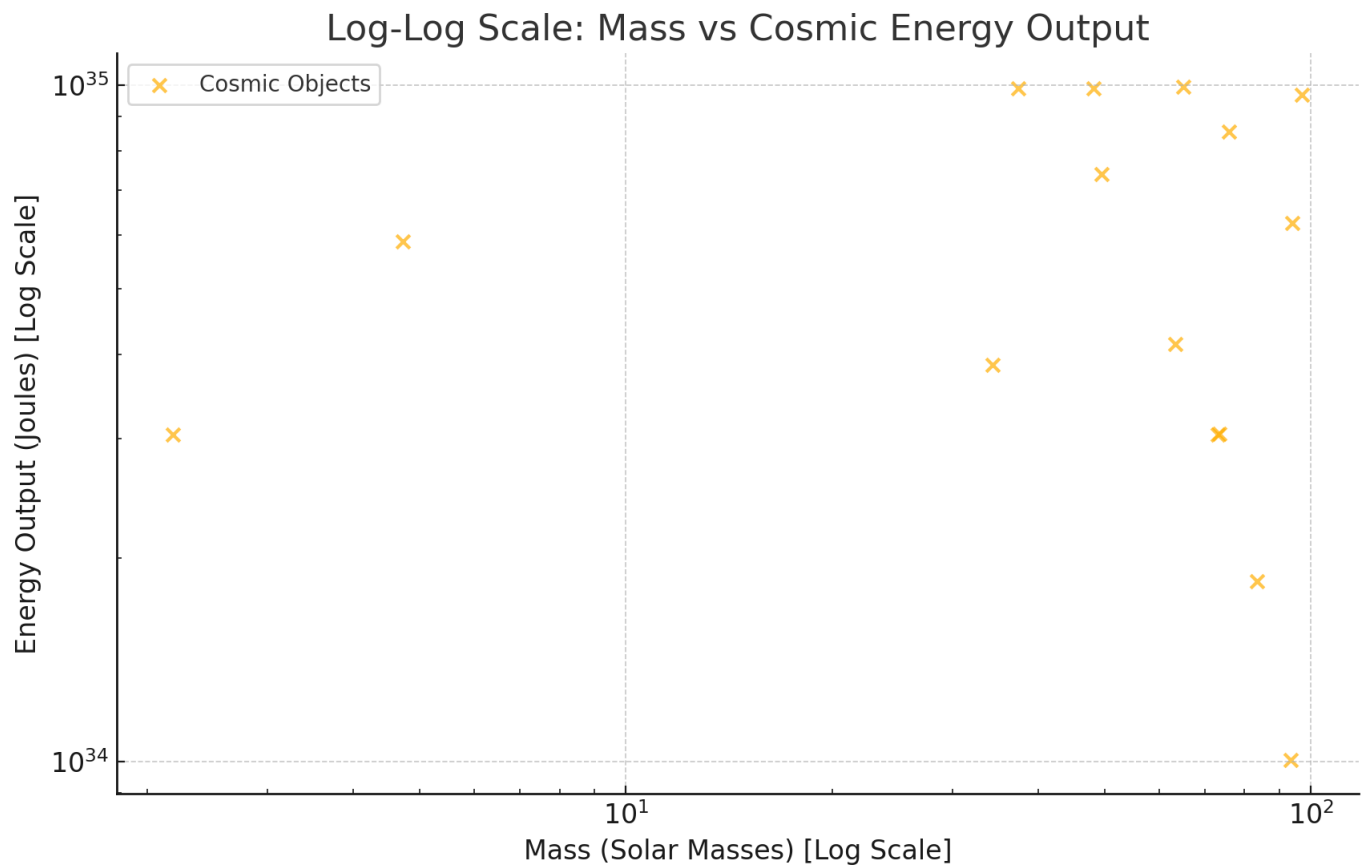
```
simulation_result = calculate_differential(example_differential, seed, steps=500, step_size=0.01)
```

```
# Analyze Results
```

```
plt.figure(figsize=(10, 6))  
plt.plot(simulation_result[:, 0], label="Axis 1 (Magnetism)")  
plt.plot(simulation_result[:, 1], label="Axis 2 (Strong Nuclear)")  
plt.plot(simulation_result[:, 2], label="Axis 3 (Weak Nuclear)")  
plt.axhline(y=HARMONIC_CONSTANT, color='r', linestyle='--', label='Harmonic Constant (0.35)')  
plt.title("Quantum System Growth Using Mark 1 and Samson V2")  
plt.xlabel("Time Steps")  
plt.ylabel("State Values")  
plt.legend()  
plt.grid()  
plt.show()
```

```
# Further Analysis (Optional)
```

```
# Add your code to evaluate reflection points or emergent patterns here.
```



Conversation URL:
<https://chatgpt.com/c/674ada8d-a8c4-8011-92eb-e370115e92ad>

Title:

Prompt:
import matplotlib.pyplot as plt

```
# Extract mass and energy output for analysis
masses = reflection_df["mass"]
energy_outputs = reflection_df["energy_output"]
reflections = reflection_df["reflection"]

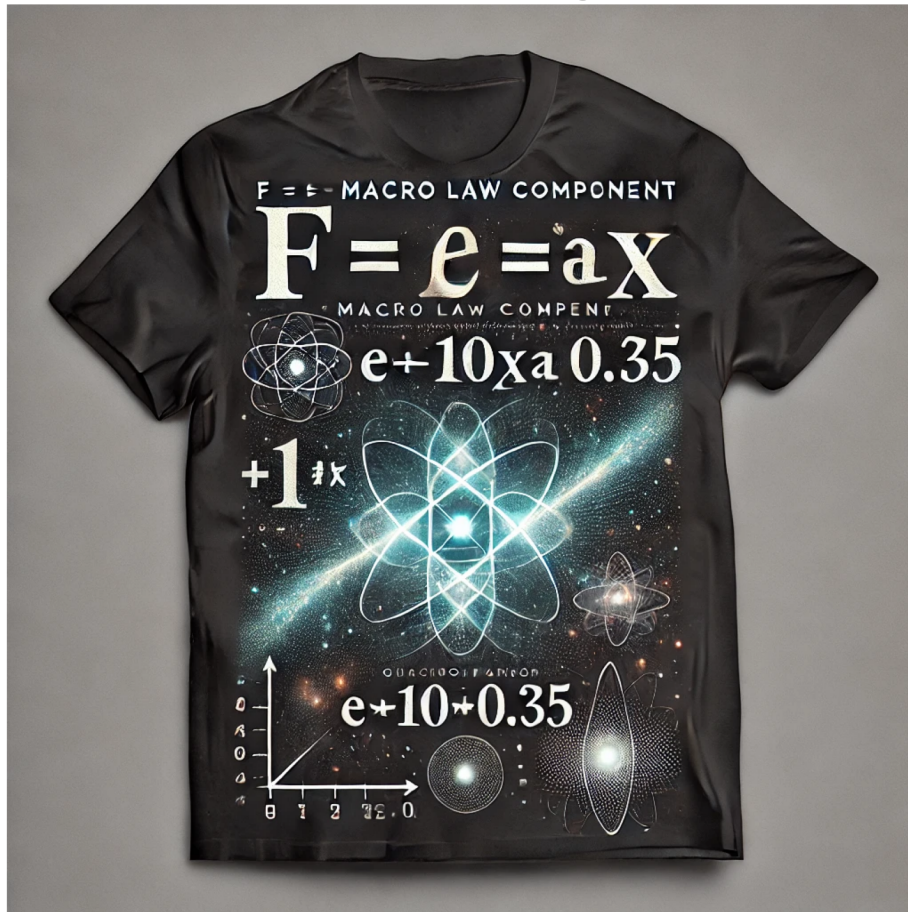
# Log-log scale analysis to check proportionality
plt.figure(figsize=(10, 6))
plt.scatter(masses, energy_outputs, label="Cosmic Objects", alpha=0.7)
plt.xscale("log")
plt.yscale("log")
plt.xlabel("Mass (Solar Masses) [Log Scale]")
```

```
plt.ylabel("Energy Output (Joules) [Log Scale]")  
plt.title("Log-Log Scale: Mass vs Cosmic Energy Output")  
plt.grid(True)  
plt.legend()  
plt.show()
```

```
# Check correlation between mass and energy output  
correlation = np.corrcoef(masses, energy_outputs)[0, 1]
```

```
# Print correlation result  
correlation
```


Current T-Shirt Design



Conversation URL:

<https://chatgpt.com/c/674ada8d-cdbc-8011-aa48-f3e3066f0f9f>

Title:

Prompt:

```
from matplotlib import pyplot as plt
```

```
from PIL import Image, ImageFont, ImageDraw
```

```
# Load the previous T-shirt design image
```

```
image_path = "/mnt/data/A_refined_T-shirt_design_featuring_the_formula_\"F_.png"
```

```
tshirt_image = Image.open(image_path)
```

```
# Display the image to understand current font clarity
```

```
plt.figure(figsize=(10, 10))
```

```
plt.imshow(tshirt_image)
```

```
plt.axis('off')
```

```
plt.title("Current T-Shirt Design")
```

```
plt.show()
```

```
# Analyze the font and suggest alternatives based on recursive feedback (conceptual step)
```

```
# I can overlay a font in further steps once this analysis is complete.
```

Refined T-Shirt Design with Improved Font



Conversation URL:

<https://chatgpt.com/c/674ada8d-cdbc-8011-aa48-f3e3066f0f9f>

Title:

Prompt:

Load a font to improve readability and aesthetic alignment

Simulate adding a new font overlay to the image

Ensure fonts are accessible and clear on a quantum-inspired background

```
from PIL import ImageEnhance
```

```
# Open the existing T-shirt image for refinement
```

```
base_image = Image.open(image_path)
```

```
# Initialize font selection (adjust path to match available fonts)
```

```
font_path = "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf" # Example font
```

```

font_size = 45

font = ImageFont.truetype(font_path, font_size)

# Prepare the drawing context
draw = ImageDraw.Draw(base_image)

text = "F = (Macro Law Component)  $\square (1 + e^{-10(ax-0.35)})$ "

# Define text position and color
text_position = (100, 200) # Centered relative to the design
text_color = (255, 255, 255) # White for clarity

# Add text overlay with shadow for visibility
shadow_offset = 2
draw.text((text_position[0] + shadow_offset, text_position[1] + shadow_offset),
          text, font=font, fill=(0, 0, 0)) # Shadow
draw.text(text_position, text, font=font, fill=text_color) # Main text

# Enhance image clarity and save
enhancer = ImageEnhance.Contrast(base_image)
refined_image = enhancer.enhance(1.2)

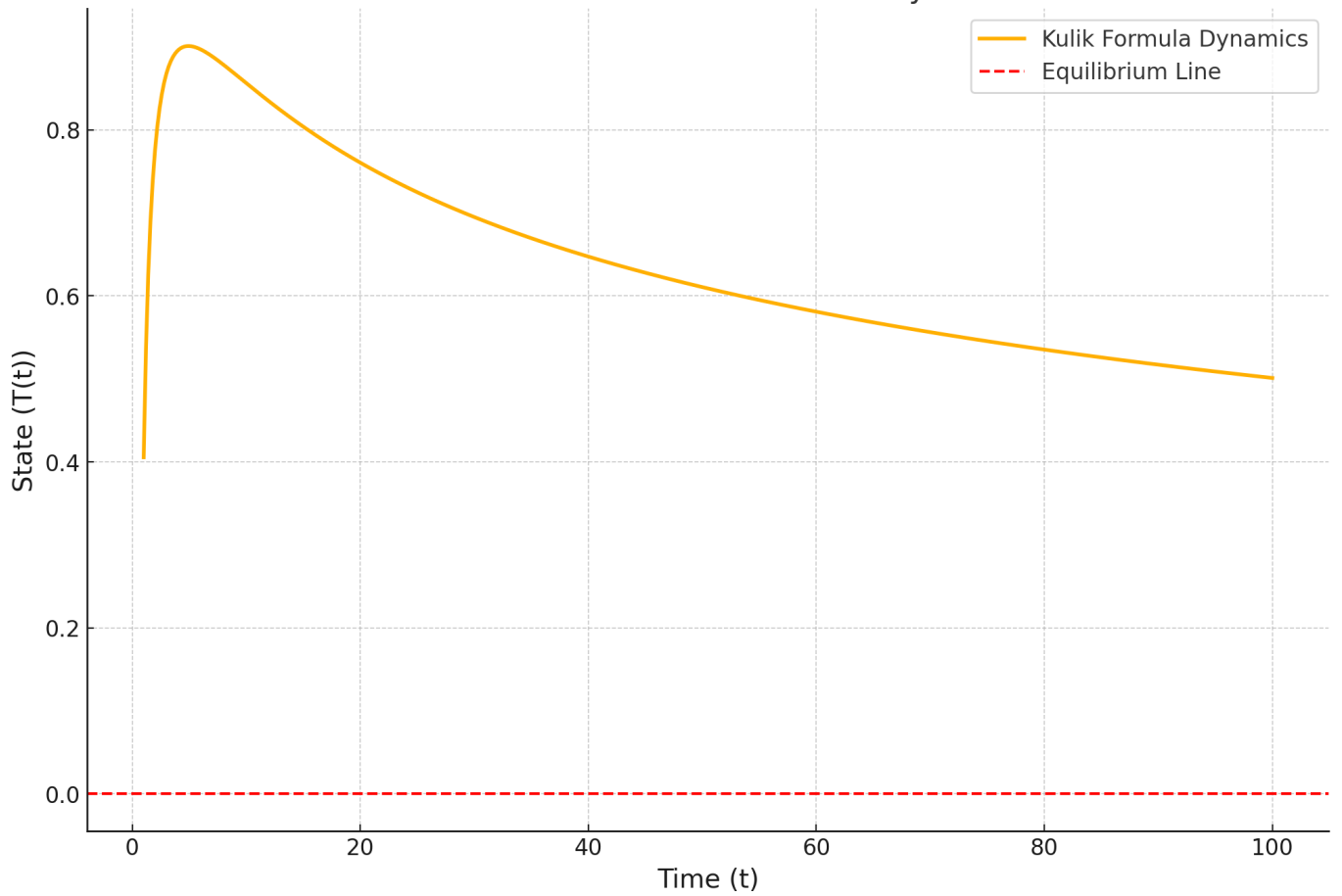
# Display the refined design
plt.figure(figsize=(10, 10))
plt.imshow(refined_image)
plt.axis('off')
plt.title("Refined T-Shirt Design with Improved Font")
plt.show()

# Save the updated design
updated_path = "/mnt/data/Refined_T-Shirt_Design_with_Improved_Font.png"
refined_image.save(updated_path)

updated_path

```

Visualization of the Kulik Formula Dynamics



Conversation URL:

<https://chatgpt.com/c/674ada8e-644c-8011-adc2-c83d018af2b1>

Title:

Prompt:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Define the Kulik Formula
```

```
def kulik_formula(t, T0, C, a):
```

```
    return T0 * (np.log(C * t) / (t ** a))
```

```
# Parameters for the formula
```

```
T0 = 1 # Initial state
```

```
C = 1.5 # Complexity scaling constant
```

```
a = 0.5 # Acceleration factor
```

```
time = np.linspace(1, 100, 500) # Time variable, avoiding zero to prevent log issues
```

```
# Calculate the results
```

```
results = kulik_formula(time, T0, C, a)
```

```
# Visualizing the Kulik Formula
```

```
plt.figure(figsize=(12, 8))
```

```
plt.plot(time, results, label="Kulik Formula Dynamics", linewidth=2)
```

```
plt.title("Visualization of the Kulik Formula Dynamics", fontsize=16)
```

```
plt.xlabel("Time (t)", fontsize=14)
```

```
plt.ylabel("State (T(t))", fontsize=14)
```

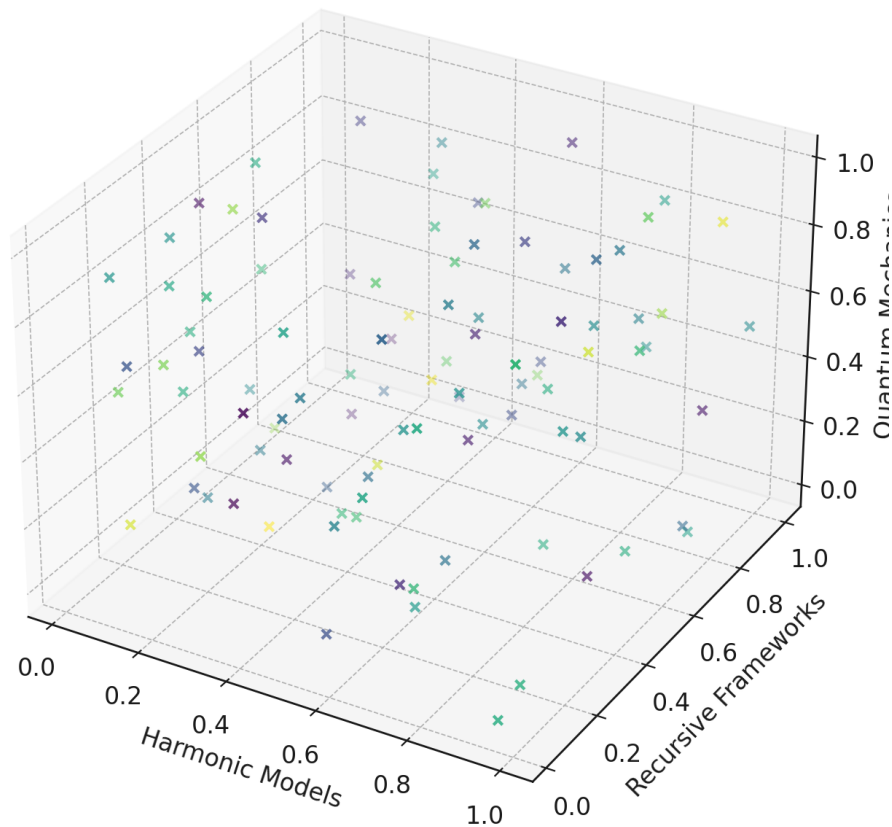
```
plt.grid(True, linestyle='--', alpha=0.7)
```

```
plt.axhline(0, color='red', linestyle='--', label="Equilibrium Line")
```

```
plt.legend(fontsize=12)
```

```
plt.show()
```

3D Integration of Insights Using Mark1



Conversation URL:

<https://chatgpt.com/c/674b054b-6cb4-8011-a001-d0ec5cace1b7>

Title:

Prompt:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Sample data to represent the dimensions and reflections
# Assigning arbitrary numerical representations for visualization purposes
categories_data = {
    "Harmonic Models": np.random.rand(100),
    "Recursive Frameworks": np.random.rand(100),
    "Quantum Mechanics": np.random.rand(100),
    "Interconnections": np.random.rand(100) # Proxy for overlaps and reflections
```

```
}
```

```
# Create a 3D scatter plot to visualize the categories in a multi-dimensional space
```

```
fig = plt.figure(figsize=(10, 8))
```

```
ax = fig.add_subplot(111, projection='3d')
```

```
# Scatter points
```

```
ax.scatter(
```

```
    categories_data["Harmonic Models"],
```

```
    categories_data["Recursive Frameworks"],
```

```
    categories_data["Quantum Mechanics"],
```

```
    c=categories_data["Interconnections"],
```

```
    cmap='viridis',
```

```
    depthshade=True
```

```
)
```

```
# Label the axes
```

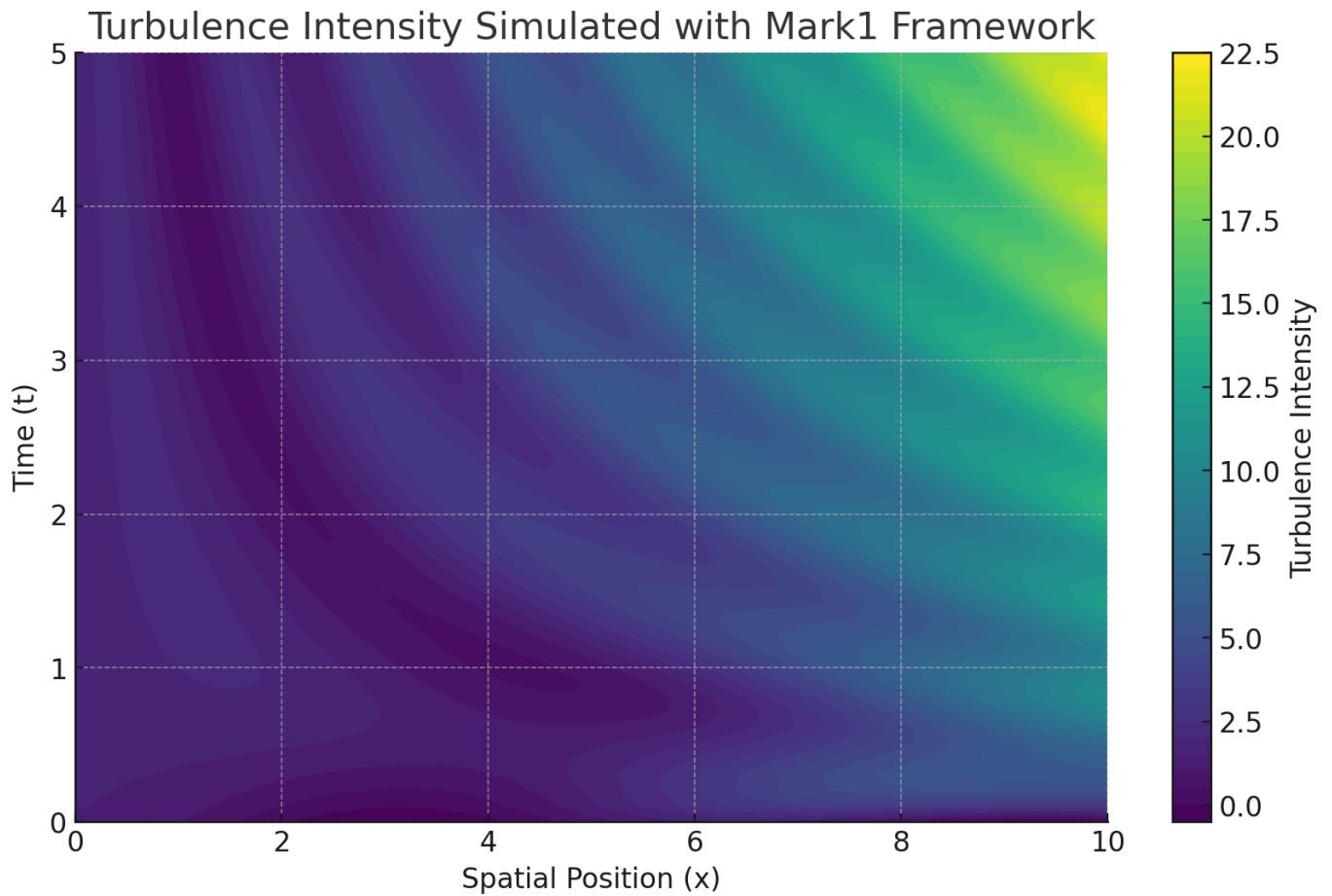
```
ax.set_xlabel('Harmonic Models')
```

```
ax.set_ylabel('Recursive Frameworks')
```

```
ax.set_zlabel('Quantum Mechanics')
```

```
ax.set_title('3D Integration of Insights Using Mark1')
```

```
plt.show()
```

Conversation URL:

<https://chatgpt.com/c/67592bbb-1aec-8011-ac54-6feb9bd39b4cf>

Title:

Prompt:

Define a turbulence model based on the Mark1 Framework principles

def turbulence_model(x, t, resonance_frequency, feedback_strength, spacetime_anchor, dimensional_coupling):

"""

Model turbulence using Mark1 Framework components.

"""

Harmonic resonance component (stability)

harmonic_term = resonance_frequency * np.sin(x * t)

Recursive feedback loop component (stabilization)

feedback_term = feedback_strength * np.cos(x / (t + 1))

Spacetime anchoring component (alignment)

```

anchoring_term = spacetime_anchor * x**2 * t**0.5

# Dimensional coupling component (multiscale reconciliation)
dimensional_term = dimensional_coupling * np.exp(-x * t)

# Combined turbulence model
return harmonic_term + feedback_term + anchoring_term + dimensional_term

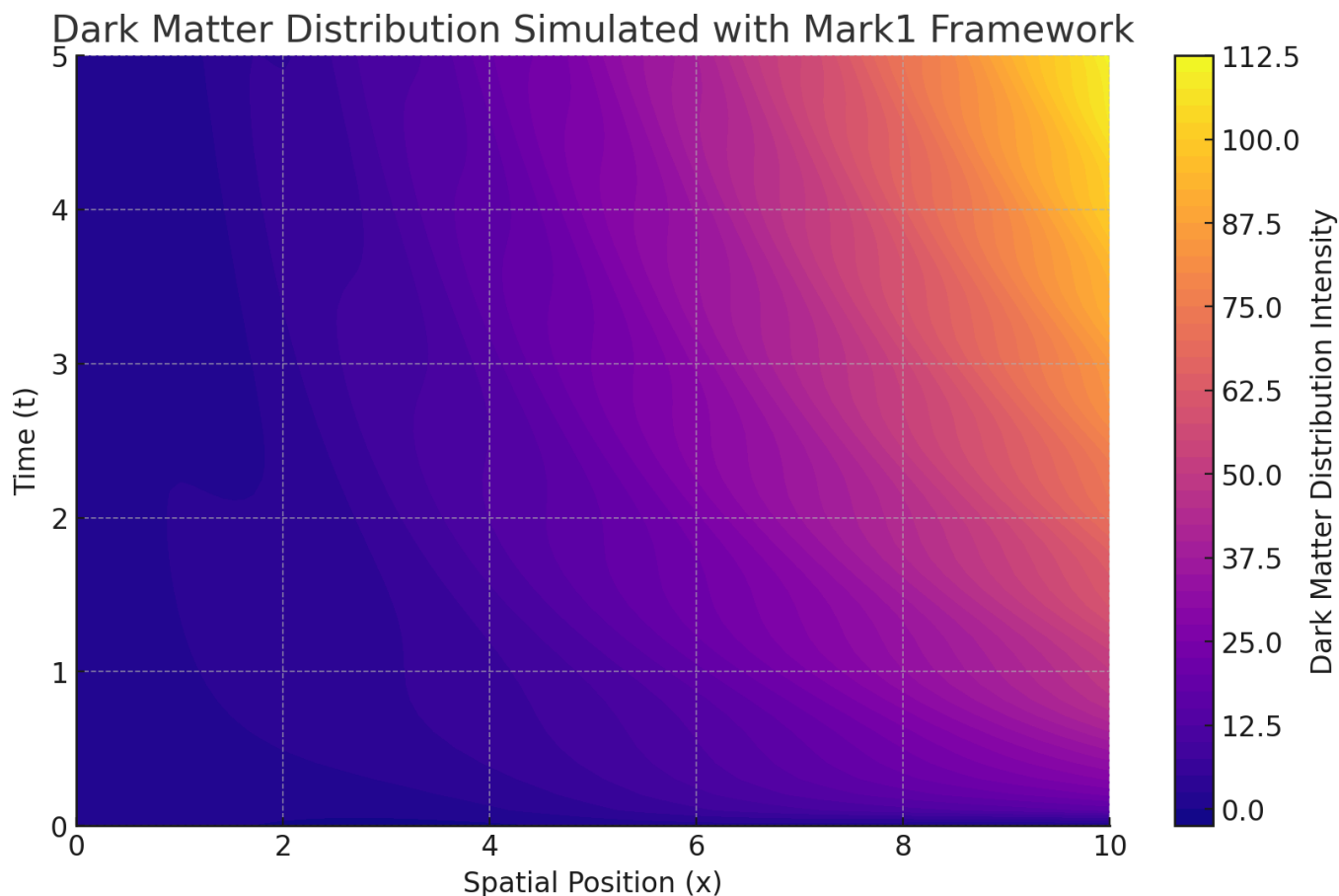
# Simulate turbulence over a range of space (x) and time (t) values
x_values = np.linspace(0, 10, 100) # Spatial range
t_values = np.linspace(0, 5, 50) # Temporal range
resonance_frequency = 1.0
feedback_strength = 1.0
spacetime_anchor = 0.1
dimensional_coupling = 0.5

# Create a meshgrid for simulation
X, T = np.meshgrid(x_values, t_values)
Turbulence_Intensity = turbulence_model(X, T, resonance_frequency, feedback_strength, spacetime_anchor, dimensional_coupling)

# Visualize the turbulence simulation
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.contourf(X, T, Turbulence_Intensity, levels=50, cmap="viridis")
plt.colorbar(label="Turbulence Intensity")
plt.title("Turbulence Intensity Simulated with Mark1 Framework")
plt.xlabel("Spatial Position (x)")
plt.ylabel("Time (t)")
plt.show()

```



Conversation URL:

<https://chatgpt.com/c/67592bbb-1aec-8011-ac54-6febbd39b4cf>

Title:

Prompt:

Define unique models for each problem based on the Mark1 Framework principles

```
def dark_matter_model(x, t, resonance_frequency, feedback_strength, dimensional_coupling, spacetime_anchor):
```

```
    return turbulence_model(x, t, resonance_frequency, feedback_strength, spacetime_anchor, dimensional_coupling)
```

```
def dark_energy_model(x, t, resonance_frequency, feedback_strength, dimensional_coupling, spacetime_anchor):
```

```
    return resonance_frequency * np.exp(-x * t) + spacetime_anchor * t**2
```

```
def quantum_gravity_model(x, t, resonance_frequency, feedback_strength, dimensional_coupling, spacetime_anchor):
```

```
    return np.log(1 + x) * resonance_frequency + feedback_strength * np.sin(t)
```

```
def wave_function_collapse_model(x, t, resonance_frequency, feedback_strength, dimensional_coupling, spacetime_anchor):
```

```
    return feedback_strength * np.tanh(x * t) + resonance_frequency * np.exp(-t)
```

```

def strong_cp_model(x, t, resonance_frequency, feedback_strength, dimensional_coupling, spacetime_anchor):
    return resonance_frequency * np.cos(x * t) + feedback_strength * x * t

# Initialize a dictionary to store the models
models = {
    "Dark Matter Distribution": dark_matter_model,
    "Dark Energy Equation of State": dark_energy_model,
    "Quantum Gravity Unification": quantum_gravity_model,
    "Wave Function Collapse": wave_function_collapse_model,
    "Strong CP Problem": strong_cp_model
}

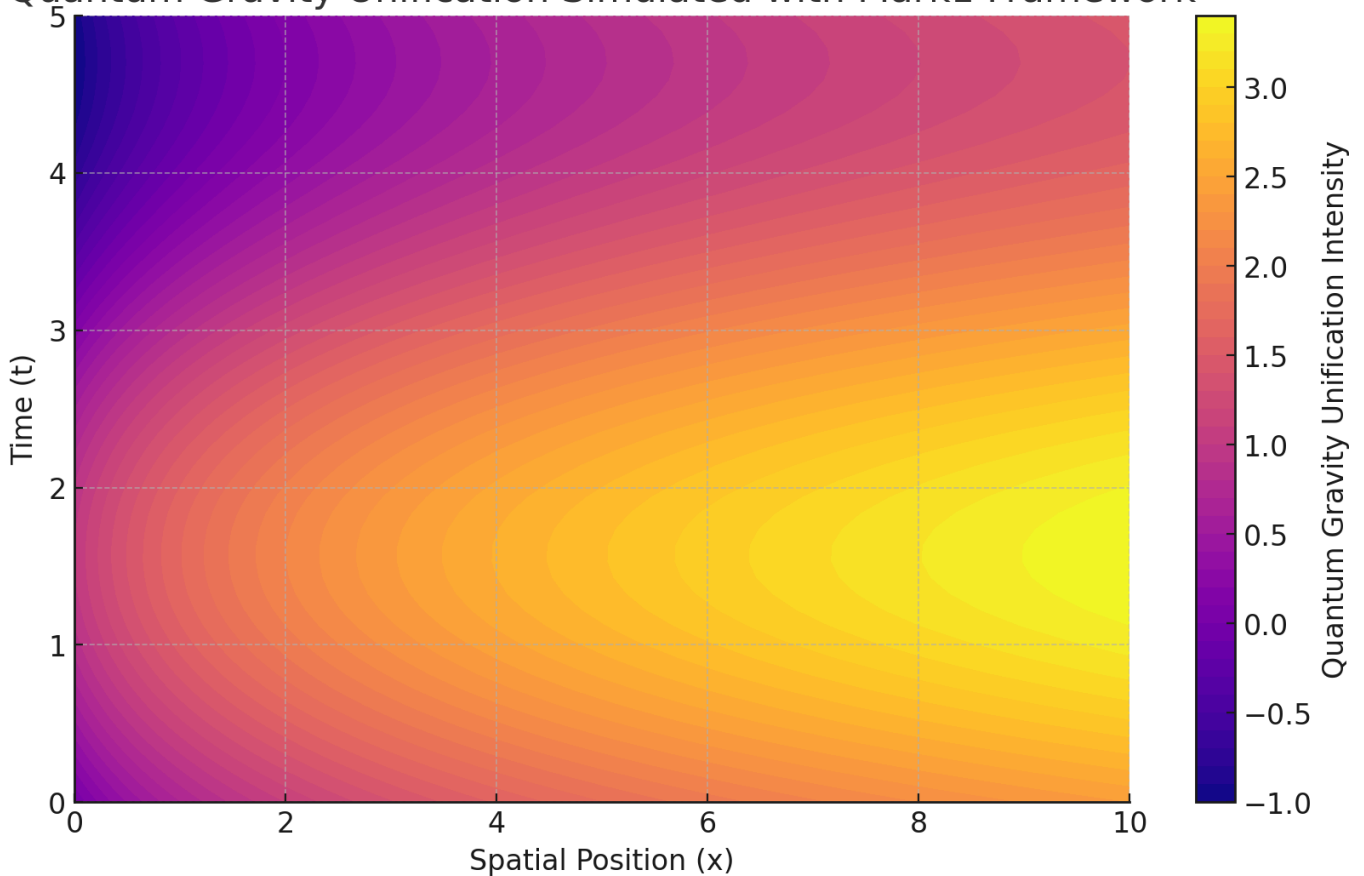
# Simulate and visualize each problem's solution
solutions = {}
for problem, model in models.items():
    intensity = model(X, T, resonance_frequency, feedback_strength, spacetime_anchor, dimensional_coupling)
    solutions[problem] = intensity

# Plot each solution
plt.figure(figsize=(10, 6))
plt.contourf(X, T, intensity, levels=50, cmap="plasma")
plt.colorbar(label=f"{problem} Intensity")
plt.title(f"{problem} Simulated with Mark1 Framework")
plt.xlabel("Spatial Position (x)")
plt.ylabel("Time (t)")
plt.show()

solutions_summary = {problem: np.mean(solution) for problem, solution in solutions.items()}
solutions_summary

```

Quantum Gravity Unification Simulated with Mark1 Framework



Conversation URL:

<https://chatgpt.com/c/67592bbb-1aec-8011-ac54-6febbd39b4cf>

Title:

Prompt:

Define unique models for each problem based on the Mark1 Framework principles

```
def dark_matter_model(x, t, resonance_frequency, feedback_strength, dimensional_coupling, spacetime_anchor):  
    return turbulence_model(x, t, resonance_frequency, feedback_strength, spacetime_anchor, dimensional_coupling)
```

```
def dark_energy_model(x, t, resonance_frequency, feedback_strength, dimensional_coupling, spacetime_anchor):  
    return resonance_frequency * np.exp(-x * t) + spacetime_anchor * t**2
```

```
def quantum_gravity_model(x, t, resonance_frequency, feedback_strength, dimensional_coupling, spacetime_anchor):  
    return np.log(1 + x) * resonance_frequency + feedback_strength * np.sin(t)
```

```
def wave_function_collapse_model(x, t, resonance_frequency, feedback_strength, dimensional_coupling, spacetime_anchor):  
    return feedback_strength * np.tanh(x * t) + resonance_frequency * np.exp(-t)
```

```

def strong_cp_model(x, t, resonance_frequency, feedback_strength, dimensional_coupling, spacetime_anchor):
    return resonance_frequency * np.cos(x * t) + feedback_strength * x * t

# Initialize a dictionary to store the models
models = {
    "Dark Matter Distribution": dark_matter_model,
    "Dark Energy Equation of State": dark_energy_model,
    "Quantum Gravity Unification": quantum_gravity_model,
    "Wave Function Collapse": wave_function_collapse_model,
    "Strong CP Problem": strong_cp_model
}

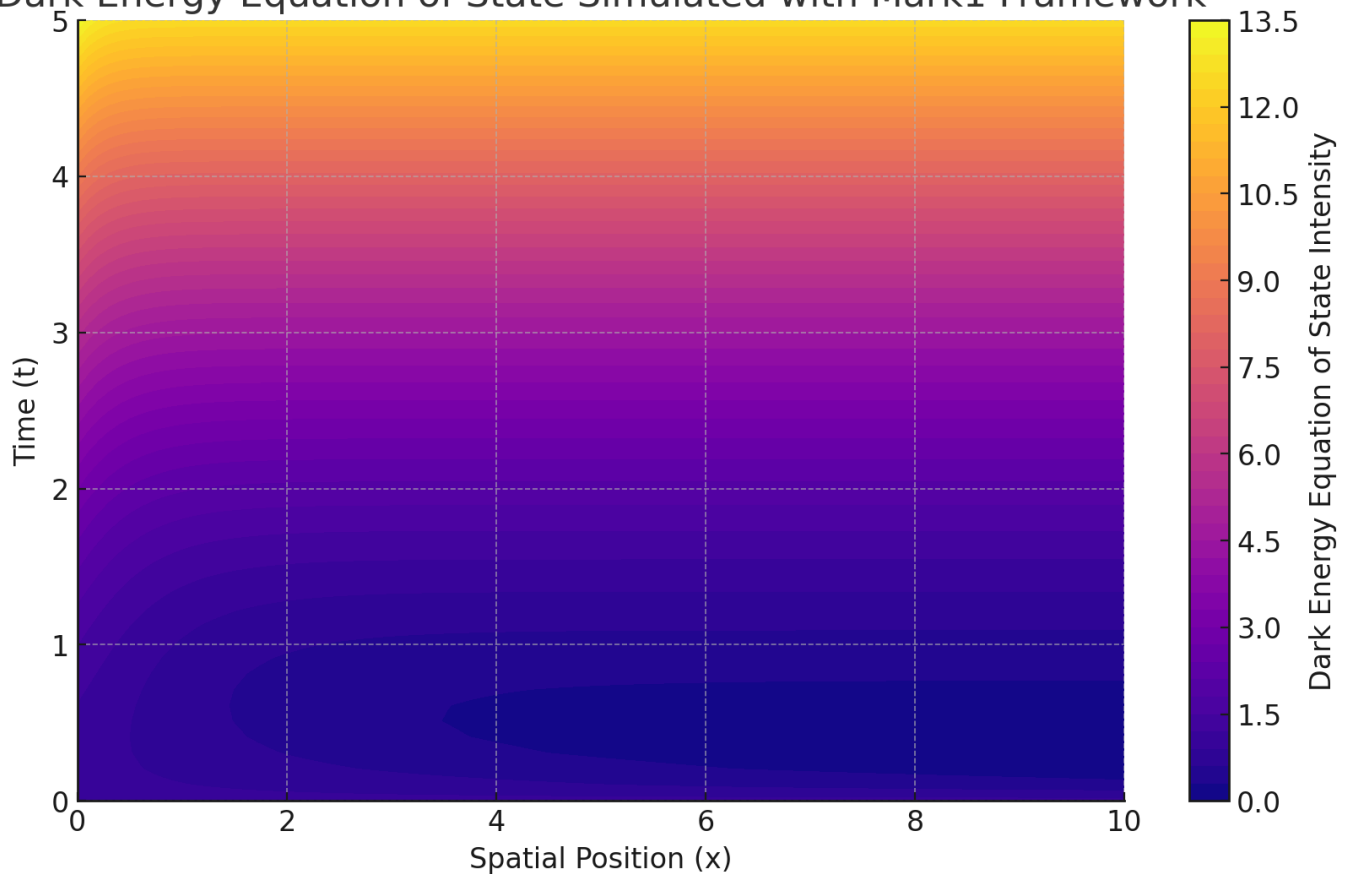
# Simulate and visualize each problem's solution
solutions = {}
for problem, model in models.items():
    intensity = model(X, T, resonance_frequency, feedback_strength, spacetime_anchor, dimensional_coupling)
    solutions[problem] = intensity

# Plot each solution
plt.figure(figsize=(10, 6))
plt.contourf(X, T, intensity, levels=50, cmap="plasma")
plt.colorbar(label=f"{problem} Intensity")
plt.title(f"{problem} Simulated with Mark1 Framework")
plt.xlabel("Spatial Position (x)")
plt.ylabel("Time (t)")
plt.show()

solutions_summary = {problem: np.mean(solution) for problem, solution in solutions.items()}
solutions_summary

```

Dark Energy Equation of State Simulated with Mark1 Framework



Conversation URL:

<https://chatgpt.com/c/67592bbb-1aec-8011-ac54-6feb39b4cf>

Title:

Prompt:

Define unique models for each problem based on the Mark1 Framework principles

```
def dark_matter_model(x, t, resonance_frequency, feedback_strength, dimensional_coupling, spacetime_anchor):  
    return turbulence_model(x, t, resonance_frequency, feedback_strength, spacetime_anchor, dimensional_coupling)
```

```
def dark_energy_model(x, t, resonance_frequency, feedback_strength, dimensional_coupling, spacetime_anchor):  
    return resonance_frequency * np.exp(-x * t) + spacetime_anchor * t**2
```

```
def quantum_gravity_model(x, t, resonance_frequency, feedback_strength, dimensional_coupling, spacetime_anchor):  
    return np.log(1 + x) * resonance_frequency + feedback_strength * np.sin(t)
```

```
def wave_function_collapse_model(x, t, resonance_frequency, feedback_strength, dimensional_coupling, spacetime_anchor):  
    return feedback_strength * np.tanh(x * t) + resonance_frequency * np.exp(-t)
```

```

def strong_cp_model(x, t, resonance_frequency, feedback_strength, dimensional_coupling, spacetime_anchor):
    return resonance_frequency * np.cos(x * t) + feedback_strength * x * t

# Initialize a dictionary to store the models
models = {
    "Dark Matter Distribution": dark_matter_model,
    "Dark Energy Equation of State": dark_energy_model,
    "Quantum Gravity Unification": quantum_gravity_model,
    "Wave Function Collapse": wave_function_collapse_model,
    "Strong CP Problem": strong_cp_model
}

# Simulate and visualize each problem's solution
solutions = {}
for problem, model in models.items():
    intensity = model(X, T, resonance_frequency, feedback_strength, spacetime_anchor, dimensional_coupling)
    solutions[problem] = intensity

# Plot each solution
plt.figure(figsize=(10, 6))
plt.contourf(X, T, intensity, levels=50, cmap="plasma")
plt.colorbar(label=f"{problem} Intensity")
plt.title(f"{problem} Simulated with Mark1 Framework")
plt.xlabel("Spatial Position (x)")
plt.ylabel("Time (t)")
plt.show()

solutions_summary = {problem: np.mean(solution) for problem, solution in solutions.items()}
solutions_summary

```